BOLU ABANT IZZET BAYSAL UNIVERSITY

DEPARTMENT OF MATHEMATICS

Graduation Project II - Apartment and Site Management System

Prepared by: Hasibe Nida Akdoğan

Student No: 210204004

Advisor: Uğur Ustaoğlu

Date: January 2026

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1. Project Purpose and Scope

This graduation project is developed to automate the financial management processes of multi-unit residential complexes and apartment sites, which constitute the core of modern urbanization. The primary motivation of the project stems from the limitations encountered in traditional methods—such as physical ledgers or basic Excel spreadsheets—including data loss, incorrect debt calculations, and difficulties in retrospective tracking. The scope of the project encompasses the management of resident information, block structures, dues (aidat) and fuel definitions, and the detailed financial reporting of all these processes through a digital interface integrated with a PostgreSQL database.

1.2. Problem Definition and Proposed Solution

The most significant challenge in current management systems is tracking which specific month or debt item a payment corresponds to. Balance discrepancies frequently occur, particularly in the accounts of residents who make irregular payments. This problem is fundamentally resolved by the **FIFO (First-In, First-Out)** based collection algorithm implemented in this project. The system ensures financial consistency by automatically allocating incoming payments to the oldest outstanding debts. Furthermore, a smart **CSV Data Migration** module has been developed to minimize human error associated with manual data entry, reducing processes that would typically take hours to mere seconds.

CHAPTER 2: TECHNOLOGICAL INFRASTRUCTURE AND ARCHITECTURE

2.1. Programming Language: Python

The core business logic (backend) and data processing algorithms of the project are built entirely on the **Python 3.x**language. The strategic reasons for selecting Python for this project are as follows:

- **Flexible Software Architecture:** It offers a highly readable syntax that minimizes the margin of error in financial calculations such as FIFO and debt restructuring.
- **Rich Library Ecosystem:** It enables different layers, such as database connectivity (Psycopg2), data analysis (Pandas), and interface development (Streamlit), to operate harmoniously within a single language environment.
- **Modularity:** Developing the application by dividing it into functional modules facilitates the future expansion and scalability of the project.

2.2. Database Management System: PostgreSQL

PostgreSQL, an enterprise-level relational database management system, was preferred for the data layer of the system.

- **Data Integrity:** By utilizing "Foreign Key" and "Unique Constraint" structures, orphaned records are prevented when a unit is deleted, ensuring 100% consistency at the database level.
- **Complex Query Performance:** The heavy data traffic between `transactions`, `units`, and `residents` tables has been optimized using SQL JOIN and aggregation queries.
- **Atomic Transactions (ACID):** Financial transactions—such as processing payments or deleting debts—are governed by ACID principles, ensuring that operations are either fully completed or rolled back in case of an error, thus preventing data corruption.

2.3. User Interface: Streamlit Framework

The frontend layer of the application was developed using **Streamlit**, a modern and reactive framework within the Python ecosystem.

- **Reactive User Experience:** It ensures that relevant metrics, such as the cash balance, are updated instantly without requiring a full page refresh after a transaction is recorded.

- **Single Page Application (SPA) Architecture:** It provides a fluid user experience through tabs and dialog windows (`@st.dialog`), simplifying complex accounting operations into a user-friendly web interface.
- **Data-Driven Design:** It enhances the transparency of the management panel by enabling the direct visualization of financial tables and charts through Python code.

2.4. Data Analysis and Processing (Pandas)

Pandas, the most powerful data analysis library in the Python ecosystem, was utilized for financial calculations and data transformation processes. Pandas performs the following critical roles:

- **Flexible Data Structures (DataFrame):** SQL query results and external CSV files are processed using Pandas `DataFrame` structures. This allows for high-speed, in-memory filtering, sorting, and grouping of thousands of rows.
- **Smart CSV Integration:** As implemented in the `toplu_yakit_yukle` module, complex debt lists in Excel/CSV formats are integrated via the `pd.read_csv()` function. Data cleaning (handling NaN values, correcting characters) and type conversions are fully automated.
- **Dynamic Filtering and Mapping:** Instead of re-querying the database, instant filtering is performed on existing datasets using Pandas' `loc` and `iloc` methods. Additionally, the mapping of unit numbers to database IDs is executed via high-performance `merge` and `join` capabilities.
- **Statistical Reporting:** Critical financial metrics are calculated and presented to the manager using aggregation functions such as `sum()`, `mean()`, and `groupby()`.

Technical Note: The integration of Pandas allows the application to function not only as a "recording tool" but also as a "data analysis tool." This enables management to analyze retrospective payment trends and financial risks in a data-driven manner.

CHAPTER 3: DATABASE DESIGN AND MODELING

3.1. Entity-Relationship Diagram (ERD) Logic

The database architecture of the system has been designed in accordance with the **Third Normal Form (3NF)** principles to eliminate data redundancy and ensure logical consistency. The core entities of the model are as follows:

- **Blocks:** Represents the physical layout and structural organization of the residential complex.
- **Units:** Contains data for independent sections (apartments) associated with specific blocks.
- **Residents:** Stores personal and contact information of the individuals residing in the units.
- **Transactions / Debt Items:** The primary tables where all financial movements, including accruals (debts) and collections (payments), are recorded.

3.2. Database Table Structures and Data Types

The following SQL structures form the backbone of the system. These schemas are optimized to maintain relational integrity and ensure the accuracy of data types across the entire lifecycle of the application:

```
/*
   FILE NAME / DOSYA ADI : revised_create_tables.sql
   PROJECT / PROJE      : Sellable Site Management Accounting Interface
                          Satılabilir Site Yönetimi Muhasebe Arayüzü

   DESCRIPTION / AÇIKLAMA:
   This script creates all core tables required for a site/apartment
   management and accounting system.

   Bu script, site/apartman yönetimi ve muhasebe sistemi için
   gerekli olan tüm temel tabloları oluşturur.
*/

-- ============================================================
-- 0. CLEANUP STEP / TEMİZLİK ADIMI
-- ============================================================
-- Drops existing tables in dependency order.
-- Tablolar, bağımlılık sırasına göre silinir.


DROP TABLE IF EXISTS payment_debt;
```

```sql
DROP TABLE IF EXISTS payment;
DROP TABLE IF EXISTS debt_item;
DROP TABLE IF EXISTS account_transaction; -- renamed from "transaction"
DROP TABLE IF EXISTS app_user;            -- renamed from "user"
DROP TABLE IF EXISTS unit;
DROP TABLE IF EXISTS unit_type;
DROP TABLE IF EXISTS building;
DROP TABLE IF EXISTS complex_properties;

--üstteki hali hata verirse
--alttakini çalıştır

-- Her şeyi kökten ve güvenli bir şekilde silmek için:
DROP TABLE IF EXISTS payment_debt CASCADE;
DROP TABLE IF EXISTS payment CASCADE;
DROP TABLE IF EXISTS debt_item CASCADE;
DROP TABLE IF EXISTS account_transaction CASCADE;
DROP TABLE IF EXISTS "transaction" CASCADE; -- Eski tablo ismi
DROP TABLE IF EXISTS app_user CASCADE;
DROP TABLE IF EXISTS "user" CASCADE;         -- Eski tablo ismi
DROP TABLE IF EXISTS unit CASCADE;
DROP TABLE IF EXISTS unit_type CASCADE;
DROP TABLE IF EXISTS building CASCADE;
DROP TABLE IF EXISTS complex_properties CASCADE;
-------------------------------------------------------------
-- 1. STRUCTURAL AND PARAMETER TABLES
-- 1. YAPISAL VE PARAMETRE TABLOLARI
-------------------------------------------------------------
/*
    TABLE / TABLO : complex_properties

    PURPOSE / AMAÇ:
    Stores general configuration settings of a site/complex.
    Siteye ait genel ayarları tutar.

    Examples / Örnekler:
    - Site name
    - Total number of units
    - Currency
*/
CREATE TABLE complex_properties (
    id SERIAL PRIMARY KEY,                      -- Unique site identifier / Site
benzersiz kimliği
    name VARCHAR(100) NOT NULL UNIQUE,           -- Site name / Site adı
    total_units INTEGER DEFAULT 0,               -- Total apartments / Toplam daire
sayısı
    currency VARCHAR(5) NOT NULL DEFAULT 'TRY',  -- Currency type / Para birimi
    dues_collection_day INTEGER DEFAULT 1        -- Monthly dues day / Aidat
toplama günü
);
```

```sql
/*
    TABLE / TABLO : building

    PURPOSE / AMAÇ:
    Represents each building or block inside a site.
    Site içerisindeki her bir bina veya bloğu temsil eder.
*/
CREATE TABLE building (
    id SERIAL PRIMARY KEY,
    complex_id INTEGER REFERENCES complex_properties(id)
        ON DELETE CASCADE NOT NULL,               -- Related site / Bağlı olduğu
site
    name VARCHAR(50) NOT NULL,                    -- Building name / Blok adı
    UNIQUE (complex_id, name)                     -- Unique per site / Site bazlı
benzersiz
);

/*
    TABLE / TABLO : unit_type

    PURPOSE / AMAÇ:
    Defines apartment types such as 2+1, 3+1.
    2+1, 3+1 gibi daire tiplerini tanımlar.

    NOTE / NOT:
    complex_id enables multi-tenancy support.
    complex_id çoklu site desteği sağlar.
*/
CREATE TABLE unit_type (
    id SERIAL PRIMARY KEY,
    complex_id INTEGER REFERENCES complex_properties(id)
        ON DELETE CASCADE NOT NULL,               -- Site reference / Site
referansı
    name VARCHAR(50) NOT NULL,                    -- Type name / Daire tipi
    default_dues NUMERIC(10,2) NOT NULL,          -- Default monthly dues /
Varsayılan aidat
    UNIQUE (complex_id, name)                     -- Scoped uniqueness / Site bazlı
benzersizlik
);

/*
    TABLE / TABLO : unit

    PURPOSE / AMAÇ:
    Represents an individual apartment.
    Her bir daireyi temsil eder.

    NOTE / NOT:
    unit_number is VARCHAR to support formats like A-1, B-10.
*/
CREATE TABLE unit (
```

```sql
    id SERIAL PRIMARY KEY,
    building_id INTEGER REFERENCES building(id)
        ON DELETE RESTRICT NOT NULL,              -- Building reference / Blok
referansı
    unit_type_id INTEGER REFERENCES unit_type(id)
        ON DELETE RESTRICT NOT NULL,              -- Apartment type / Daire tipi
    unit_number VARCHAR(20) NOT NULL,             -- Apartment number / Daire
numarası
    owner_name VARCHAR(100) NOT NULL,             -- Owner name / Ev sahibi
    tenant_name VARCHAR(100),                     -- Tenant name / Kiracı
    phone VARCHAR(20),                            -- Contact phone / Telefon
    UNIQUE (building_id, unit_number)             -- Unique per building / Blok
bazlı benzersiz
);

-------------------------------------------------------------
-- 2. ACCOUNTING AND MANAGEMENT TABLES
-- 2. MUHASEBE VE YÖNETİM TABLOLARI
-------------------------------------------------------------
/*
    TABLE / TABLO : app_user

    PURPOSE / AMAÇ:
    Stores application users.
    Sistem kullanıcılarını tutar.

    NOTE / NOT:
    Renamed to avoid SQL reserved keyword "user".
*/
CREATE TABLE app_user (
    id SERIAL PRIMARY KEY,
    complex_id INTEGER REFERENCES complex_properties(id)
        ON DELETE SET NULL,                       -- Assigned site / Bağlı site
    unit_id INTEGER REFERENCES unit(id)
        ON DELETE SET NULL,                       -- Related unit / İlgili daire
    username VARCHAR(50) NOT NULL UNIQUE,         -- Login username / Kullanıcı adı
    password_hash VARCHAR(255) NOT NULL,          -- Password hash / Şifre özeti
    role VARCHAR(20) NOT NULL
        CHECK (role IN ('MANAGER', 'RESIDENT'))   -- User role / Kullanıcı rolü
);
/*
    TABLE / TABLO : account_transaction

    PURPOSE / AMAÇ:
    Stores income and expense records.
    Gelir ve gider işlemlerini tutar.

    NOTE / NOT:
    created_at is used for audit tracking.
*/
CREATE TABLE account_transaction (
```

```sql
    id SERIAL PRIMARY KEY,
    complex_id INTEGER REFERENCES complex_properties(id)
        ON DELETE RESTRICT NOT NULL,              -- Site reference / Site
referansı
    type VARCHAR(10) NOT NULL
        CHECK (type IN ('INCOME', 'EXPENSE')),    -- Transaction type / İşlem türü
    category VARCHAR(50) NOT NULL,                -- Category / Kategori
    unit_id INTEGER REFERENCES unit(id)
        ON DELETE SET NULL,                       -- Related unit / İlgili daire
    amount NUMERIC(10,2) NOT NULL,                -- Amount / Tutar
    process_date TIMESTAMP NOT NULL,              -- Effective date / İşlem tarihi
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, -- Insert timestamp / Kayıt
zamanı
    description TEXT                              -- Description / Açıklama
);

/*
    TABLE / TABLO : debt_item

    PURPOSE / AMAÇ:
    Represents expected monthly debts per unit.
    Daire bazlı aylık borçları temsil eder.
*/


CREATE TABLE debt_item (
    id SERIAL PRIMARY KEY,
    unit_id INTEGER REFERENCES unit(id)
        ON DELETE CASCADE NOT NULL,               -- Related unit / İlgili daire
    type VARCHAR(10) NOT NULL
        CHECK (type IN ('DUES', 'FUEL', 'OTHER')),-- Debt type / Borç türü
    period_month DATE NOT NULL,                   -- Debt period / Borç dönemi
    expected_amount NUMERIC(10,2) NOT NULL,       -- Expected amount / Beklenen
tutar
    status VARCHAR(20) NOT NULL
        CHECK (status IN ('UNPAID', 'PAID', 'PARTIAL')), -- Payment status / Durum
    UNIQUE (unit_id, type, period_month)
);



/*
    TABLE / TABLO : payment

    PURPOSE / AMAÇ:
    Stores received payments.
    Alınan ödemeleri tutar.

*/

CREATE TABLE payment (
```

```
    id SERIAL PRIMARY KEY,
    complex_id INTEGER REFERENCES complex_properties(id)
        ON DELETE RESTRICT NOT NULL,              -- Site reference / Site
referansı
    unit_id INTEGER REFERENCES unit(id)
        ON DELETE RESTRICT NOT NULL,              -- Paying unit / Ödeme yapan
daire
    amount NUMERIC(10,2) NOT NULL,                -- Paid amount / Ödenen tutar
    process_date TIMESTAMP NOT NULL,              -- Receipt date / Dekont tarihi
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, -- Entry time / Giriş zamanı
    reference_no VARCHAR(50),                     -- Reference number / Referans no
    description TEXT                              -- Description / Açıklama
);

/*
    TABLE / TABLO : payment_debt

    PURPOSE / AMAÇ:
    Creates a many-to-many relationship between payments and debts.
    Ödemeler ile borçlar arasında çoktan-çoğa ilişki kurar.
*/

CREATE TABLE payment_debt (
    id SERIAL PRIMARY KEY,
    payment_id INTEGER REFERENCES payment(id)
        ON DELETE CASCADE NOT NULL,               -- Payment reference / Ödeme
referansı
    debt_item_id INTEGER REFERENCES debt_item(id)
        ON DELETE CASCADE NOT NULL,               -- Debt reference / Borç
referansı
    covered_amount NUMERIC(10,2) NOT NULL,        -- Covered amount / Karşılanan
tutar
    UNIQUE (payment_id, debt_item_id)
);
------------------------------
/*
    TABLE: unit
    PURPOSE (EN): Indicates whether a unit (apartment) is exempt from payments
                  such as dues or fees.
    PURPOSE (TR): Dairenin aidat veya benzeri ödemelerden muaf olup olmadığını
                  belirtir.
*/
ALTER TABLE unit

ADD COLUMN IF NOT EXISTS is_exempt BOOLEAN DEFAULT FALSE;
```

[SQL Code for Table Creation Omitted - Corresponds to source 106-349]

3.3. Database Triggers and Functions

The logical structures that ensure the automation of the system and execute in the background during financial transactions are detailed below. In particular, the management of collection processes and real-time balance updates is handled directly at the database level through these triggers, ensuring data consistency across all financial tables.

```
/*
   FILE NAME / DOSYA ADI : create_payment_transaction_trigger.sql
   PROJECT / PROJE      : Sellable Site Management Accounting Interface
                          Satılabilir Site Yönetimi Muhasebe Arayüzü

   PURPOSE / AMAÇ:
   EN: Automatically creates an INCOME transaction record whenever a new payment is
inserted.
       This ensures that the accounting ledger (account_transaction) stays
consistent with
       payment records (payment).

   TR: payment tablosuna yeni bir ödeme kaydı eklendiğinde, otomatik olarak
       account_transaction tablosuna bir GELİR (INCOME) hareketi ekler.
       Böylece ödeme kayıtları ile muhasebe kayıtları senkron kalır.

   WHEN IT RUNS? / NE ZAMAN ÇALIŞIR?
   EN: After a row is inserted into the payment table (AFTER INSERT).
   TR: payment tablosuna yeni satır eklendikten sonra (AFTER INSERT).

   NOTES / NOTLAR:
   - EN: Implemented as a PostgreSQL trigger + PL/pgSQL function.
   - TR: PostgreSQL trigger + PL/pgSQL fonksiyonu kullanılarak uygulanmıştır.
*/

-------------------------------------------------------------
-- 1) TRIGGER FUNCTION / TETİKLEYİCİ FONKSİYON
-------------------------------------------------------------
/*
   EN:
   This function is executed by the trigger after a payment insert.
   It inserts a corresponding row into account_transaction as an INCOME.

   TR:
   Bu fonksiyon, payment tablosuna ödeme eklendikten sonra tetikleyici tarafından
çalıştırılır.
   account_transaction tablosuna INCOME türünde bir kayıt ekler.

   INPUT / GİRDİ:
   - NEW: The newly inserted payment row (payment tablosuna eklenen yeni satır)

   OUTPUT / ÇIKTI:
```

```
   - Returns NEW so that the original insert operation proceeds normally.
   - NEW döndürülür; ödeme ekleme işlemi normal şekilde devam eder.
*/
CREATE OR REPLACE FUNCTION fn_create_payment_transaction()
RETURNS TRIGGER AS $$
BEGIN
    /*
      EN:
      Insert an accounting transaction that mirrors the payment record.

      TR:
      Ödeme kaydını yansıtan bir muhasebe hareketi oluşturulur.
    */
    INSERT INTO account_transaction (
        complex_id,      -- EN: Which site/complex this belongs to / TR: Hangi
siteye ait
        type,            -- EN: Transaction type (INCOME/EXPENSE) / TR: İşlem türü
        category,        -- EN: Sub-category for reporting / TR: Raporlama
kategorisi
        unit_id,         -- EN: Related apartment/unit / TR: İlgili daire
        amount,          -- EN: Monetary amount / TR: Tutar
        process_date,    -- EN: Effective date of the transaction / TR: İşlem
tarihi
        description      -- EN: Human-readable description / TR: Açıklama metni
    )
    VALUES (
        NEW.complex_id,                -- EN/TR: from payment row
        'INCOME',                      -- EN: Always income for received payments
                                       -- TR: Daireden alınan ödeme = gelir
        'PAYMENT_RECEIVED',            -- EN: Category for traceability/reporting
                                       -- TR: İzlenebilirlik/raporlama kategorisi
        NEW.unit_id,                   -- EN/TR: paying unit
        NEW.amount,                    -- EN/TR: paid amount
        NEW.process_date,              -- EN/TR: payment process date

        /*
          EN:
          Build a description string. If NEW.description is NULL,
          COALESCE converts it to empty string to avoid NULL concatenation issues.

          TR:
          Açıklama metni oluşturulur. NEW.description NULL ise,
          COALESCE bunu boş string'e çevirerek birleştirme hatasını önler.
        */
        'Daireden gelen ödeme: ' || COALESCE(NEW.description, '')
    );
```

```
    /*
      EN:
      Return NEW to confirm trigger execution and keep the original INSERT intact.

      TR:
      NEW döndürülür; tetikleyici başarılı çalışır ve ödeme INSERT işlemi bozulmaz.
    */
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- ------------------------------------------------------------
-- 2) TRIGGER DEFINITION / TETİKLEYİCİ TANIMI
-- ------------------------------------------------------------
/*
   EN:
   This trigger fires AFTER INSERT on payment table, for each inserted row,
   and executes fn_create_payment_transaction().

   TR:
   Bu tetikleyici payment tablosunda AFTER INSERT olayında,
   eklenen her satır için çalışır ve fn_create_payment_transaction() fonksiyonunu
çağırır.
*/
CREATE TRIGGER trg_after_payment_insert
AFTER INSERT ON payment
FOR EACH ROW
EXECUTE FUNCTION fn_create_payment_transaction();
```

[SQL Code for Triggers Omitted - Corresponds to source 353-471]

3.4. Initial Data Configuration

The core definitions required for the application to become operational (such as block names, apartment types, and debt categories) are integrated into the system through this configuration script. This process ensures that the fundamental database environment is pre-populated with the necessary static data before the system goes live.

```
/*
   FILE NAME / DOSYA ADI : insert_initial_data.sql

   PURPOSE / AMAÇ:
   EN: Inserts all initial (seed) data fully compatible with the revised schema
       (app_user, account_transaction, etc.). The script includes all 56 apartments
       belonging to the site.

   TR: Yeni şemaya (app_user, account_transaction vb.) tamamen uyumlu olacak
şekilde
       başlangıç (örnek) verilerini ekler. Script, sitedeki toplam 56 dairenin
       tamamını içermektedir.

   NOTES / NOTLAR:
   - EN: This script assumes that table creation scripts have already been
executed.
   - TR: Bu script, tablo oluşturma scriptlerinin daha önce çalıştırıldığını
varsayar.
*/


-------------------------------------------------------------
-- 1. SITE INFORMATION / SİTE BİLGİSİ
-------------------------------------------------------------
/*

   EN:
   Inserts the main site/complex information.

   TR:
   Ana site (kompleks) bilgileri eklenir.

*/


INSERT INTO complex_properties (name, total_units, currency, dues_collection_day)
VALUES (
    'DUZCE ILI KAYNASLI ILCESI KARACALI MAHALLESI TOPLU KONUTLARI',
    56,        -- EN/TR: Total apartment count / Toplam daire sayısı
```

```
    'TRY',     -- EN/TR: Currency / Para birimi
    1          -- EN/TR: Monthly dues collection day / Aidat toplama günü
);



-- ------------------------------------------------------------
-- 2. BUILDINGS / BLOKLAR
-- ------------------------------------------------------------
/*
   EN:
   Inserts all buildings belonging to the site.
   Each building is linked using complex_id = 1.

   TR:
   Siteye ait tüm bloklar eklenir.
   Her blok complex_id = 1 olacak şekilde ilişkilendirilmiştir.
*/
INSERT INTO building (complex_id, name) VALUES
(1, 'KT1'), (1, 'KT2'), (1, 'KT3'),
(1, 'KT4'), (1, 'KT5'), (1, 'KT6'), (1, 'KT7');

-- ------------------------------------------------------------
-- 3. APARTMENT TYPES / DAİRE TİPLERİ
-- ------------------------------------------------------------
/*
   EN:
   Defines apartment types for the site.
   Both apartment types have the same default dues amount.

   TR:
   Siteye ait daire tipleri tanımlanır.
   Her iki daire tipi için varsayılan aidat aynıdır.
*/
INSERT INTO unit_type (complex_id, name, default_dues) VALUES
(1, '2+1', 800.00),
(1, '3+1', 800.00);

-- ------------------------------------------------------------
-- 4. MANAGER ACCOUNT / YÖNETİCİ HESABI
-- ------------------------------------------------------------
/*
   EN:
   Inserts a management (administrator) user.
   This user is responsible for accounting and system operations.

   TR:
   Muhasebe ve sistem işlemlerinden sorumlu yönetici kullanıcı eklenir.
*/
INSERT INTO app_user (complex_id, username, password_hash, role)
VALUES (
    1,
```

```sql
    'muhasebe_yoneticisi',
    'cok_guclu_hash_12345', -- EN/TR: Placeholder password hash / Örnek şifre hash'i
    'MANAGER'
);

-------------------------------------------------------------
-- 5. ALL APARTMENTS (56 UNITS) / TÜM DAİRELER (56 ADET)
-------------------------------------------------------------
/*
   EN:
   Inserts all apartment records grouped by building.
   Each building contains 8 apartments.

   TR:
   Tüm daire kayıtları blok bazlı olarak eklenir.
   Her blokta 8 daire bulunmaktadır.
*/

-- KT1 (building_id = 1)
INSERT INTO unit (building_id, unit_type_id, unit_number, owner_name) VALUES
(1, 1, '1', 'MURAT ÖZÇELİK'), (1, 2, '2', 'HALENUR DEMİREL'),
(1, 1, '3', 'AZMİ KAYACI'),   (1, 2, '4', 'TAYFUN ARAPOĞLU'),
(1, 1, '5', 'FATİH ERCAN'),   (1, 2, '6', 'YAKUP AKYÜZ'),
(1, 1, '7', 'ASUMAN MADENCİOĞLU'), (1, 2, '8', 'NİRGÜL KUMAŞ');

-- KT2 (building_id = 2)
INSERT INTO unit (building_id, unit_type_id, unit_number, owner_name) VALUES
(2, 2, '1', 'ALİ ERDEM'), (2, 1, '2', 'CELAL ŞEN'),
(2, 2, '3', 'EMRİYE SÜRER'), (2, 1, '4', 'DURHAN BAY'),
(2, 2, '5', 'OKAN ŞENER'), (2, 1, '6', 'FİKRET DEMİREL'),
(2, 2, '7', 'KEREM VEZİROĞLU'), (2, 1, '8', 'NERMİN KAHRAMAN');

-- KT3 (building_id = 3)
INSERT INTO unit (building_id, unit_type_id, unit_number, owner_name) VALUES
(3, 2, '1', 'MURAT SAKALLI'), (3, 1, '2', 'AHMET YAP'),
(3, 2, '3', 'EMEL KURAL'), (3, 1, '4', 'GÜRDAL ÇUBUKÇU'),
(3, 2, '5', 'ÖZKAN KAHRAMAN'), (3, 1, '6', 'KAMİL ÖNAY'),
(3, 2, '7', 'SELÇUK YILDIRIM'), (3, 1, '8', 'ZAFER SAKAOĞLU');

-- KT4 (building_id = 4)
INSERT INTO unit (building_id, unit_type_id, unit_number, owner_name) VALUES
(4, 1, '1', 'EMRE AL'), (4, 2, '2', 'BEYTULLAH İNCE'),
(4, 1, '3', 'GÜLİZAR USTA'), (4, 2, '4', 'MURAT ÇİLLİ'),
(4, 1, '5', 'EMRAH DURKAN'), (4, 2, '6', 'YUSUF YASİN ÇİLLİOĞLU'),
(4, 1, '7', 'EMRAH KURT'), (4, 2, '8', 'MURAT SAY');

-- KT5 (building_id = 5)
INSERT INTO unit (building_id, unit_type_id, unit_number, owner_name) VALUES
(5, 2, '1', 'TUNCAY KADAN'), (5, 1, '2', 'MURAT KIROĞLU'),
(5, 2, '3', 'ÖMER BAYRAM'), (5, 1, '4', 'AHMET AKDOĞAN'),
```

```sql
(5, 2, '5', 'FAHRİ AKTAŞ'), (5, 1, '6', 'OKTAY DİK'),
(5, 2, '7', 'ERTUĞRUL KAYA'), (5, 1, '8', 'ERSİN TOPRAK');



-- KT6 (building_id = 6)
INSERT INTO unit (building_id, unit_type_id, unit_number, owner_name) VALUES
(6, 1, '1', 'ÇETİN ÖZKAN'), (6, 2, '2', 'MUSTAFA ALTUNDAL'),
(6, 1, '3', 'MÜCAHİT TUNÇ'), (6, 2, '4', 'HURİ PALAZOĞLU'),
(6, 1, '5', 'MEHMET YAP'), (6, 2, '6', 'MURAT ÖZKAN'),
(6, 1, '7', 'YUSUF KARAL'), (6, 2, '8', 'DAVUT ÖZTÜRK');

-- KT7 (building_id = 7)
INSERT INTO unit (building_id, unit_type_id, unit_number, owner_name) VALUES
(7, 2, '1', 'ÜMMÜHAN ARDIÇ'), (7, 1, '2', 'SAMET AYDIN'),
(7, 2, '3', 'GÜLER AKDOĞAN'), (7, 1, '4', 'FAZLI TEZCAN'),
(7, 2, '5', 'CANER YILMAZ'), (7, 1, '6', 'MURAT KOCA'),
(7, 2, '7', 'BEKİR HELİMERGÜN'), (7, 1, '8', 'RECAİ ULUIŞIK');


-------------------------------------------------------------
-- 6. ALL EMPLOYEES / TÜM PERSONELLER
-------------------------------------------------------------
/*
   EN:
   Inserts initial employee records for the housing complex.
   Includes cleaning staff, block managers, and the general site manager.
   Some management roles are voluntary, therefore their salary is set to 0.

   TR:
   Siteye ait başlangıç personel kayıtları eklenir.
   Temizlik personeli, blok yöneticileri ve site genel yöneticisi dahildir.
   Bazı yönetici görevleri gönüllülük esaslı olduğu için maaş bilgisi 0 olarak
girilmiştir.
*/

INSERT INTO employee (complex_id, name, role, salary) VALUES
(1, 'Hamiyet Özaltın', 'Temizlik Görevlisi', 4500),
(1, 'Durhan Bay', 'Yönetici (KT1-KT2)', 0),
(1, 'Beytullah İnce', 'Yönetici (KT3-KT4)', 0),
(1, 'Mustafa Altundal', 'Yönetici (KT5-KT6)', 0),
(1, 'Sadettin Akdoğan', 'Yönetici (KT7)', 0),
(1, 'Sadettin Akdoğan', 'Genel Toki Başkanı', 1500);

-------------------------------------------------------------
-- UNIT EXEMPT STATUS UPDATE / DAİRE MUAFİYET DURUMU GÜNCELLEME
-------------------------------------------------------------
/*
   EN:
   Updates the "is_exempt" field for specific unit owners.
   These units are marked as exempt from payments such as dues or fees.
```

```
   TR:
   Belirli daire sahipleri için muafiyet durumu güncellenir.
   Bu daireler aidat veya benzeri ödemelerden muaf olarak işaretlenir.
*/
UPDATE unit
SET is_exempt = TRUE
WHERE owner_name IN (
    'Durhan Bay',
    'Beytullah İnce',
    'Mustafa Altundal',
    'Sadettin Akdoğan'
);
```

[SQL Code for Initial Data Omitted - Corresponds to source 475-778]

3.5. Legacy Data Migration

This section covers the process of transferring data from physical accounting ledgers currently in active use to the digital system, apart from setting up the system from scratch.

Important Note: This SQL script is specifically configured to define the past debt balances of an existing apartment management into the system as a one-time (migration) action. Managers who will start using the application anew do not have to do this step manually; the "**Bulk Debt Entry**" and "**Data Transfer with CSV**" modules in the application's user interface (Frontend) can perform these operations without the need for writing code.

The following commands are special transfer queries prepared to ensure the consistency of real-time data:

```
/*
   FILE NAME / DOSYA ADI : inters_past_period_debts.sql
   PROJECT / PROJE      : Sellable Site Management Accounting Interface
                          Satılabilir Site Yönetimi Muhasebe Arayüzü

   PURPOSE / AMAÇ:
   EN: Inserts historical (past-period) debt records for apartments.
       This script is used to initialize dues and fuel debts that
       belong to December 2025 and earlier periods.

   TR: Dairelere ait geçmiş dönem (historical) borç kayıtlarını ekler.
       Bu script, Aralık 2025 ve öncesine ait aidat ve yakıt borçlarının
       sisteme ilk kez tanımlanması amacıyla kullanılır.

   IMPORTANT / ÖNEMLİ:
```

```
   EN: This script does NOT generate future debts.
   TR: Bu script geleceğe yönelik borç üretmez.

   NOTES / NOTLAR:
   - EN: Intended to be executed once during system setup or migration.
   - TR: Sistem kurulumu veya veri aktarımı sırasında bir defaya mahsus
çalıştırılır.
*/

-------------------------------------------------------------
-- HISTORICAL DUES DEFINITION (DECEMBER 2025)
-- GEÇMİŞ DÖNEM AİDAT BORCU TANIMLAMA (ARALIK 2025)
-------------------------------------------------------------
/*
   EN:
   Creates an UNPAID monthly dues debt for every apartment
   for December 2025.

   TR:
   Aralık 2025 dönemi için tüm dairelere ödenmemiş (UNPAID)
   aidat borcu tanımlar.
*/
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
SELECT
    id,                 -- EN/TR: Apartment identifier / Daire kimliği
    'DUES',              -- EN: Monthly dues / TR: Aidat
    '2025-12-01',        -- EN: Period start date / TR: Dönem başlangıcı
    800.00,              -- EN/TR: Dues amount / Aidat tutarı
    'UNPAID'             -- EN: Initial debt status / TR: Başlangıç durumu
FROM unit;




/* KT1 BLOĞU - ÖDENMEMİŞ GEÇMİŞ BORÇLARIN GİRİŞİ */

-- Tayfun Arapoğlu (ID: 4) - Toplam Birikmiş Borç
-- 2024-12-01 tarihini 'devir tarihi' olarak kullanıyoruz.
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (4, 'OTHER', '2024-12-01', 13468.00, 'UNPAID');

-- Fatih Ercan (ID: 5) - Kasım ve Aralık Borcu (Ekranda 1600 görünüyor)
-- Zaten Aralık aidatını (800) otomatik eklemiştik, şimdi Kasım'ı ekleyelim.
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (5, 'DUES', '2025-11-01', 800.00, 'UNPAID');

-- Yakup Akyüz (ID: 6) - Ekim, Kasım ve Aralık Borcu (Ekranda 2400 görünüyor)
-- Aralık zaten var, Ekim ve Kasım'ı ekleyelim.
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (6, 'DUES', '2025-10-01', 800.00, 'UNPAID'),
       (6, 'DUES', '2025-11-01', 800.00, 'UNPAID');
```

```sql
/* CELAL ŞEN (ID: 10) BORÇ DÜZELTME */

-- 1. Daha önce girdiğimiz 800 TL'lik Aralık aidatını 200 TL'ye indiriyoruz
UPDATE debt_item
SET expected_amount = 200.00
WHERE unit_id = 10
  AND period_month = '2025-12-01'
  AND type = 'DUES';

-- 2. Temmuz'dan Kasım sonuna kadar olan toplam 700 TL birikmiş borcu ekliyoruz
-- (Temmuz-Ağustos-Eylül: 3x100 + Ekim-Kasım: 2x200 = 700 TL)
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (10, 'OTHER', '2025-11-01', 700.00, 'UNPAID');

/* MURAT KIROĞLU (ID: 34) BORÇ DÜZELTME */

-- 1. Daha önce otomatik atanan 800 TL'lik Aralık aidatını 200 TL'ye indiriyoruz
UPDATE debt_item
SET expected_amount = 200.00
WHERE unit_id = 34
  AND period_month = '2025-12-01'
  AND type = 'DUES';

-- 2. Nisan - Kasım arası birikmiş 1000 TL borcu ekliyoruz
-- (Nisan-Eylül: 6 ay x 100 TL = 600 TL)
-- (Ekim-Kasım: 2 ay x 200 TL = 400 TL)
-- TOPLAM: 1000 TL Geçmiş Borç
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (34, 'OTHER', '2025-11-01', 1000.00, 'UNPAID');

/* KT2 BLOĞU - KALAN EKSİKLERİN TAMAMLANMASI */

-- 1. Ali Erdem (ID: 9) - Ekim ve Kasım Aidatları (Aralık zaten var)
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(9, 'DUES', '2025-10-01', 800.00, 'UNPAID'),
(9, 'DUES', '2025-11-01', 800.00, 'UNPAID');

-- 2. Okan Şener (ID: 13) - Sadece Kasım Aidatı (Aralık zaten var)
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(13, 'DUES', '2025-11-01', 800.00, 'UNPAID');

-- 3. Durhan Bay (ID: 12) - YÖNETİCİ MUAFİYETİ
-- Aralık aidatını 'ÖDENDİ' olarak işaretleyip kapatıyoruz.
UPDATE debt_item
SET status = 'PAID'
WHERE unit_id = 12
  AND period_month = '2025-12-01'
  AND type = 'DUES';
```

```sql
  /* KT4 BLOĞU - MALİ DURUM GÜNCELLEME
   (Murat Çilli'nin geçmiş borcu ve Yönetici muafiyeti dahil)
*/

-- 1. CİDDİ BİRİKMİŞ BORÇLARIN GİRİŞİ (Emrah Kurt ve Murat Say)
-- Aralık ayı aidatı (800 TL) sistemde zaten tanımlı olduğu için toplam bakiyeden 800 TL düşerek giriyoruz.
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(31, 'OTHER', '2025-11-01', 6600.00, 'UNPAID'), -- Emrah Kurt: Toplam 7400 (6600 Geçmiş + 800 Aralık)
(32, 'OTHER', '2025-11-01', 6100.00, 'UNPAID'); -- Murat Say: Toplam 6900 (6100 Geçmiş + 800 Aralık)

-- 2. MURAT ÇİLLİ (ID: 28) - GEÇMİŞTEN KALAN EKSİK BORÇ
-- Belirttiğin Eylül döneminden sarkan 500 TL'lik borcu ekliyoruz.
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (28, 'OTHER', '2025-09-01', 500.00, 'UNPAID');

-- 3. BEYTULLAH İNCE (ID: 26) - YÖNETİCİ MUAFİYETİ
-- Apartman yöneticisi olduğu için Aralık ayı borcunu "ÖDENDİ" olarak kapatıyoruz.
UPDATE debt_item
SET status = 'PAID'
WHERE unit_id = 26
  AND period_month = '2025-12-01'
  AND type = 'DUES';

  /* KT5 BLOĞU - GEÇMİŞ BORÇ GİRİŞİ */

-- Tuncay Kadan (ID: 33)
-- Toplam 3400 TL borç (Aralık 800 dahil).
-- Geçmiş borç: 3400 - 800 = 2600 TL.
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (33, 'OTHER', '2025-11-01', 2600.00, 'UNPAID');


/* KT6 BLOĞU - MALİ DURUM GÜNCELLEME */

-- 1. MEHMET YAP (ID: 45) - KASIM AİDATI EKLEME
-- Aralık aidatı (800 TL) zaten var, yanına Kasım'ı (800 TL) ekliyoruz. Toplam 1600 TL olacak.
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (45, 'DUES', '2025-11-01', 800.00, 'UNPAID');

-- 2. MUSTAFA ALTINDAL (ID: 42) - YÖNETİCİ MUAFİYETİ
-- Yönetici olduğu için Aralık aidatını "ÖDENDİ" olarak işaretliyoruz.
UPDATE debt_item
SET status = 'PAID'
WHERE unit_id = 42
  AND period_month = '2025-12-01'
  AND type = 'DUES';
```

```sql
  /* KT7 BLOĞU - YÖNETİCİ MUAFİYETİ */
UPDATE debt_item
SET status = 'PAID'
WHERE unit_id = 51
  AND period_month = '2025-12-01'
  AND type = 'DUES';


  /* 1. TÜM SİTEYE OCAK 2026 AİDATI ATAMA */
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
SELECT id, 'DUES', '2026-01-01', 800.00, 'UNPAID'
FROM unit;

/* 2. ÖZEL ANLAŞMALI DAİRELERİN DÜZELTİLMESİ (200 TL) */
UPDATE debt_item
SET expected_amount = 200.00
WHERE period_month = '2026-01-01'
  AND unit_id IN (10, 34); -- Celal Şen ve Murat Kıroğlu

/* 3. YÖNETİCİ MUAFİYETLERİNİN UYGULANMASI */
UPDATE debt_item
SET status = 'PAID'
WHERE period_month = '2026-01-01'
  AND unit_id IN (12, 26, 42, 51); -- Durhan, Beytullah, Mustafa, Güler


  /* KT1 BLOĞU - YAKIT BORÇLARI KESİN LİSTE */

-- 1. Murat Özçelik (ID: 1) - Aralık Borcu
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (1, 'FUEL', '2025-12-01', 1571.00, 'UNPAID');

-- 2. Halenur Demirel (ID: 2) - Aralık Borcu
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (2, 'FUEL', '2025-12-01', 1425.00, 'UNPAID');

-- 3. Azmi Kayacı (ID: 3) - Aralık + Eksik Kalan Bakiye (50 TL)
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(3, 'FUEL', '2025-12-01', 1194.00, 'UNPAID'),
(3, 'FUEL', '2025-11-01', 50.00, 'UNPAID'); -- Eksik ödeme bakiyesi

-- 4. Tayfun Arapoğlu (ID: 4) - Aralık + Geçen Kış Devri (5930 TL - Lacivert)
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(4, 'FUEL', '2025-12-01', 1273.00, 'UNPAID'),
(4, 'FUEL', '2024-01-01', 5930.00, 'UNPAID'); -- Gerçek geçen kış borcu

-- 5. Fatih Ercan (ID: 5) - Ekim, Kasım, Aralık
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(5, 'FUEL', '2025-10-01', 744.50, 'UNPAID'),
(5, 'FUEL', '2025-11-01', 849.31, 'UNPAID'),
(5, 'FUEL', '2025-12-01', 1194.00, 'UNPAID');
```

```sql
-- 6. Yakup Akyüz (ID: 6) - Ekim, Kasım, Aralık
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(6, 'FUEL', '2025-10-01', 876.20, 'UNPAID'),
(6, 'FUEL', '2025-11-01', 1235.00, 'UNPAID'),
(6, 'FUEL', '2025-12-01', 1699.00, 'UNPAID');

-- 8. Nurgül Kumaş (ID: 8) - Aralık Borcu
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (8, 'FUEL', '2025-12-01', 1462.00, 'UNPAID');

/* KT2 BLOĞU - YAKIT BORÇLARI (KESİN TUTARLAR) */

-- 1. Ali Erdem (ID: 9) - Ekim, Kasım, Aralık
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(9, 'FUEL', '2025-10-01', 1837.00, 'UNPAID'),
(9, 'FUEL', '2025-11-01', 1795.00, 'UNPAID'),
(9, 'FUEL', '2025-12-01', 2110.00, 'UNPAID');

-- 2. Celal Şen (ID: 10) - Ekim, Kasım, Aralık
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(10, 'FUEL', '2025-10-01', 248.00, 'UNPAID'),
(10, 'FUEL', '2025-11-01', 850.00, 'UNPAID'),
(10, 'FUEL', '2025-12-01', 1194.00, 'UNPAID');

-- 3. Emriye Sürer (ID: 11) - Sadece Aralık
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (11, 'FUEL', '2025-12-01', 1522.00, 'UNPAID');

-- 4. Durhan Bay (ID: 12) - Sadece Aralık
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (12, 'FUEL', '2025-12-01', 1043.00, 'UNPAID');

/* KT3 BLOĞU - YAKIT BORÇLARI VE EKSİK BAKİYELER (KESİN LİSTE) */

-- 1. Kamil Önay (ID: 24) - Aralık Yakıt Borcu
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (24, 'FUEL', '2025-12-01', 1172.00, 'UNPAID');

-- 2. Eksik Kalan Bakiyelerin Girilmesi
-- (Tutar ve isim eşleştirmelerini senin verdiğin verilere göre güncelledim)
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(17, 'FUEL', '2025-11-01', 35.00, 'UNPAID'),  -- Murat Sakallı (ID: 17)
(18, 'FUEL', '2025-11-01', 35.00, 'UNPAID'),  -- Ahmet Yap (ID: 18)
(19, 'FUEL', '2025-11-01', 737.00, 'UNPAID'), -- Selçuk Yıldırım (ID: 19)
(20, 'FUEL', '2025-11-01', 70.00, 'UNPAID');  -- Zafer Sakaoğlu (ID: 20)

/* KT4 BLOĞU - YAKIT BORÇLARI VE GEÇMİŞ KIŞ DEVİRLERİ */

-- 1. Küçük Bakiye Eksikleri (Ocak sütunundaki 35 TL'ler)
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(25, 'FUEL', '2025-11-01', 35.00, 'UNPAID'), -- Emre Al
```

```sql
(26, 'FUEL', '2025-11-01', 35.00, 'UNPAID'); -- Beytullah İnce

-- 2. Aylık Yakıt Borçları (Gülizar, Emrah D., Yusuf Yasin)
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(27, 'FUEL', '2025-10-01', 493.00, 'UNPAID'),  -- Gülizar Usta (Ekim)
(29, 'FUEL', '2025-12-01', 896.00, 'UNPAID'),  -- Emrah Durkan (Aralık)
(30, 'FUEL', '2025-11-01', 1284.80, 'UNPAID'), -- Yusuf Yasin (Kasım)
(30, 'FUEL', '2025-12-01', 1608.00, 'UNPAID'); -- Yusuf Yasin (Aralık)

-- 3. Emrah Kurt (ID: 31) - Ekim, Kasım, Aralık + 9332 TL Lacivert Devir
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(31, 'FUEL', '2025-10-01', 795.00, 'UNPAID'),
(31, 'FUEL', '2025-11-01', 1576.78, 'UNPAID'),
(31, 'FUEL', '2025-12-01', 1783.00, 'UNPAID'),
(31, 'FUEL', '2024-01-01', 9332.00, 'UNPAID'); -- Lacivert (Geçmiş Kış)

-- 4. Murat Say (ID: 32) - Ekim, Kasım, Aralık + 11288 TL Lacivert Devir
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(32, 'FUEL', '2025-10-01', 752.00, 'UNPAID'),
(32, 'FUEL', '2025-11-01', 1062.91, 'UNPAID'),
(32, 'FUEL', '2025-12-01', 1751.00, 'UNPAID'),
(32, 'FUEL', '2024-01-01', 11288.00, 'UNPAID'); -- Lacivert (Geçmiş Kış)

/* KT5 BLOĞU - YAKIT BORÇLARI VE ÖZEL BAKİYELER */

-- 1. Tuncay Kadan (ID: 33) - 3 Ay Ödenmemiş
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(33, 'FUEL', '2025-10-01', 646.50, 'UNPAID'),
(33, 'FUEL', '2025-11-01', 1007.00, 'UNPAID'),
(33, 'FUEL', '2025-12-01', 1515.00, 'UNPAID');

-- 2. Murat Kıroğlu (ID: 34) - Aralık + 500 TL Gider/Bakiye
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(34, 'FUEL', '2025-12-01', 1315.00, 'UNPAID'),
(34, 'FUEL', '2025-11-01', 500.00, 'UNPAID'); -- '500 gider' notuna istinaden

-- 3. Fahri Aktaş (ID: 37) - Aralık + 128 TL Borç
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(37, 'FUEL', '2025-12-01', 1515.00, 'UNPAID'),
(37, 'FUEL', '2025-11-01', 128.00, 'UNPAID');

-- 4. Ertuğrul Kaya (ID: 39) - Sadece Aralık
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (39, 'FUEL', '2025-12-01', 1515.00, 'UNPAID');
/* KT6 BLOĞU - MEHMET YAP VE YUSUF KARAL YAKIT GÜNCELLEMESİ */

-- 1. MEHMET YAP (ID: 45) Borçları
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(45, 'FUEL', '2025-11-01', 807.00, 'UNPAID'),
(45, 'FUEL', '2025-12-01', 1315.00, 'UNPAID'),
(45, 'FUEL', '2025-11-15', 500.00, 'UNPAID'); -- '500 Gider' borcu
```

```sql
-- MEHMET YAP (ID: 45) 300 TL Artı Bakiye (Gelecek borçtan düşmek üzere)
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (45, 'FUEL', '2025-12-31', -300.00, 'PAID');


-- 2. YUSUF KARAL (ID: 47) Borçları
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(47, 'FUEL', '2025-11-01', 807.00, 'UNPAID'),
(47, 'FUEL', '2025-12-01', 1315.00, 'UNPAID');
/* KT7 BLOĞU - YAKIT BORÇLARI GİRİŞİ */

-- 1. Samet Aydın (ID: 50) - Aralık
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (50, 'FUEL', '2025-12-01', 1463.00, 'UNPAID');

-- 2. Caner Yılmaz (ID: 53) - Aralık
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (53, 'FUEL', '2025-12-01', 1663.00, 'UNPAID');

-- 3. Bekir Helimergün (ID: 55) - Kasım ve Aralık
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status) VALUES
(55, 'FUEL', '2025-11-01', 1269.00, 'UNPAID'),
(55, 'FUEL', '2025-12-01', 1663.00, 'UNPAID');

-- 4. Recai Uluışık (ID: 56) - Aralık
INSERT INTO debt_item (unit_id, type, period_month, expected_amount, status)
VALUES (56, 'FUEL', '2025-12-01', 1463.00, 'UNPAID');

/* 1. AZMİ KAYACI (ID: 3) - ELDEN ÖDEME */
-- Kasaya girmemesi için direkt borç tablosunu güncelliyoruz
UPDATE debt_item
SET status = 'PAID'
WHERE unit_id = 3
  AND period_month = '2025-12-01'
  AND type = 'DUES';

/* 2. BANKAYA YATAN DİĞER ÖDEMELER */
-- Hem created_at hem de process_date sütunlarını dolduruyoruz
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description) VALUES
(1, 1, 800.00, '2025-12-07', '2025-12-07', 'Aralık 2025 Aidat Ödemesi'),
(1, 2, 800.00, '2025-12-05', '2025-12-05', 'Aralık 2025 Aidat Ödemesi'),
(1, 7, 800.00, '2025-12-23', '2025-12-23', 'Aralık 2025 Aidat Ödemesi'),
(1, 8, 800.00, '2025-12-05', '2025-12-05', 'Aralık 2025 Aidat Ödemesi');

/* KT2 BLOĞU - ARALIK 2025 AİDAT TAHSİLATLARI */

-- 1. Fikret Demirel (ID: 13)
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
```

```
VALUES (1, 13, 800.00, '2025-12-02', '2025-12-02', 'Aralık 2025 Aidat Ödemesi');

-- 2. Kerem Veziroğlu (ID: 14)
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 14, 800.00, '2025-12-25', '2025-12-25', 'Aralık 2025 Aidat Ödemesi');

-- 3. Nermin Kahraman (ID: 15)
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 15, 800.00, '2025-12-06', '2025-12-06', 'Aralık 2025 Aidat Ödemesi');

/* KT3 BLOĞU - ARALIK 2025 AİDAT TAHSİLATLARI */

-- 1. Murat Sakallı (ID: 17)
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 17, 800.00, '2025-12-06', '2025-12-06', 'Aralık 2025 Aidat Ödemesi');

-- 2. Ahmet Yap (ID: 18)
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 18, 800.00, '2025-12-26', '2025-12-26', 'Aralık 2025 Aidat Ödemesi');

-- 3. Emel Kural (ID: 19)
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 19, 800.00, '2025-12-12', '2025-12-12', 'Aralık 2025 Aidat Ödemesi');

-- 4. Gürdal Çubukçu (ID: 20)
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 20, 800.00, '2025-12-03', '2025-12-03', 'Aralık 2025 Aidat Ödemesi');

-- 5. Özkan Kahraman (ID: 21)
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 21, 800.00, '2025-12-26', '2025-12-26', 'Aralık 2025 Aidat Ödemesi');

-- 6. Kamil Önay (ID: 22)
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 22, 800.00, '2025-12-01', '2025-12-01', 'Aralık 2025 Aidat Ödemesi');

-- 7. Selçuk Yıldırım (ID: 23) - Erken Ödeme (Temmuz)
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 23, 800.00, '2025-07-05', '2025-07-05', 'Aralık 2025 Aidat Ödemesi
(Peşin/Avans)');

-- 8. Zafer Sakaoğlu (ID: 24) - Erken Ödeme (Kasım)
```

```sql
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 24, 800.00, '2025-11-05', '2025-11-05', 'Aralık 2025 Aidat Ödemesi (Ön
Ödeme)');
/* Beytullah İnce (ID: 26) - Yönetici Muafiyeti Uygulaması */
UPDATE debt_item
SET status = 'PAID'
WHERE unit_id = 26
  AND type = 'DUES'
  AND period_month = '2025-12-01';

/* KT4 BLOĞU - ARALIK 2025 AİDAT TAHSİLATLARI */

-- 1. Emre Al (ID: 25)
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 25, 800.00, '2025-12-15', '2025-12-15', 'Aralık 2025 Aidat Ödemesi');

-- 2. Gülizar Usta (ID: 27)
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 27, 800.00, '2025-12-15', '2025-12-15', 'Aralık 2025 Aidat Ödemesi');

-- 3. Murat Çilli (ID: 28)
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 28, 800.00, '2025-12-02', '2025-12-02', 'Aralık 2025 Aidat Ödemesi');

-- 4. Yusuf Yasin Çillioğlu (ID: 30)
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 30, 800.00, '2025-12-04', '2025-12-04', 'Aralık 2025 Aidat Ödemesi');

/* KT5 BLOĞU - ARALIK 2025 AİDAT TAHSİLATLARI */

-- 1. Ahmet Akdoğan (ID: 36)
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 36, 800.00, '2025-12-13', '2025-12-13', 'Aralık 2025 Aidat Ödemesi');

-- 2. Fahri Aktaş (ID: 37)
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 37, 800.00, '2025-12-09', '2025-12-09', 'Aralık 2025 Aidat Ödemesi');

-- 3. Oktay Dik (ID: 38)
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 38, 800.00, '2025-12-19', '2025-12-19', 'Aralık 2025 Aidat Ödemesi');

-- 4. Ertuğrul Kaya (ID: 39)
```

```sql
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 39, 800.00, '2025-12-03', '2025-12-03', 'Aralık 2025 Aidat Ödemesi');

-- 5. Ersin Toprak (ID: 40)
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 40, 800.00, '2025-12-07', '2025-12-07', 'Aralık 2025 Aidat Ödemesi');

/* KT6 BLOĞU TAHSİLATLARI VE MUAFİYETLER */

-- Mustafa Altındal (ID: 42) - Muafiyet (Kasaya para girmeden borç kapatma)
UPDATE debt_item SET status = 'PAID' WHERE unit_id = 42 AND type = 'DUES' AND
period_month = '2025-12-01';

-- KT6 Ödemeleri
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description) VALUES
(1, 41, 800.00, '2025-12-23', '2025-12-23', 'Aralık 2025 Aidat Ödemesi'), -- Çetin
Özkan
(1, 43, 800.00, '2025-12-21', '2025-12-21', 'Aralık 2025 Aidat Ödemesi'), --
Mücahit Tunç
(1, 44, 800.00, '2025-12-15', '2025-12-15', 'Aralık 2025 Aidat Ödemesi'), -- Huri
Palazoğlu
(1, 46, 800.00, '2025-12-15', '2025-12-15', 'Aralık 2025 Aidat Ödemesi'), -- Murat
Özkan
(1, 47, 800.00, '2025-11-14', '2025-11-14', 'Aralık 2025 Aidat (Ön Ödeme)'), --
Yusuf Karal
(1, 48, 800.00, '2025-11-16', '2025-11-16', 'Aralık 2025 Aidat (Ön Ödeme)'); --
Davut Öztürk

/* KT7 BLOĞU TAHSİLATLARI VE MUAFİYETLER */

-- Güler Akdoğan (ID: 51) - Muafiyet
UPDATE debt_item SET status = 'PAID' WHERE unit_id = 51 AND type = 'DUES' AND
period_month = '2025-12-01';

-- KT7 Ödemeleri
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description) VALUES
(1, 49, 800.00, '2025-12-21', '2025-12-21', 'Aralık 2025 Aidat Ödemesi'), --
Ümmühan Ardıç
(1, 50, 800.00, '2025-12-26', '2025-12-26', 'Aralık 2025 Aidat Ödemesi'), -- Samet
Aydın
(1, 52, 800.00, '2025-12-25', '2025-12-25', 'Aralık 2025 Aidat Ödemesi'), -- Fazlı
Tezcan
(1, 53, 800.00, '2025-12-10', '2025-12-10', 'Aralık 2025 Aidat Ödemesi'), -- Caner
Yıldırım
(1, 54, 800.00, '2025-11-21', '2025-11-21', 'Aralık 2025 Aidat (Ön Ödeme)'), --
Murat Koca
```

```
(1, 55, 800.00, '2025-12-01', '2025-12-01', 'Aralık 2025 Aidat Ödemesi'); -- Bekir
Helimergün


/* 1. CELAL ŞEN'İN TÜM BORÇLARINI ÖDENDİ OLARAK GÜNCELLE */
UPDATE debt_item
SET status = 'PAID'
WHERE unit_id = 10
  AND status = 'UNPAID'
  AND period_month <= '2025-12-01';

/* 2. TAHSİLAT KAYDINI OLUŞTUR (Trigger sayesinde kasaya otomatik işlenir) */
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 10, 3200.00, CURRENT_DATE, CURRENT_DATE, 'Celal Şen - Tüm borçlar
kapatıldı (Aralık dahil)');

/* 1. EMRIYE SÜRER (ID: 11) BORÇLARINI 'ÖDENDİ' YAP */
UPDATE debt_item
SET status = 'PAID'
WHERE unit_id = 11
  AND status = 'UNPAID'
  AND period_month <= '2025-12-01';

/* 2. TAHSİLAT KAYDINI GİR (2320 TL) */
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 11, 2320.00, CURRENT_DATE, CURRENT_DATE, 'Emriye Sürer - Aralık borçları
kapatıldı');


--kt5d7
/* 1. TÜM BORÇLARI (ARALIK 2025 VE OCAK 2026) 'ÖDENDİ' YAP */
UPDATE debt_item
SET status = 'PAID'
WHERE unit_id = 39
  AND status = 'UNPAID'
  AND (period_month = '2025-12-01' OR period_month = '2026-01-01');

/* 2. TOPLAM TAHSİLAT KAYDINI GİR (3115 TL) */
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 39, 3115.00, '2025-12-28', '2025-12-28', 'Ertuğrul Kaya - Aralık
Yakıt+Aidat ve Ocak Aidat Peşin Ödemesi');


/* 1. YAKUP AKYÜZ (ID: 6) TÜM GEÇMİŞ BORÇLARI 'ÖDENDİ' YAP */
UPDATE debt_item
SET status = 'PAID'
WHERE unit_id = 6
  AND status = 'UNPAID'
  AND period_month <= '2025-12-01';
```

```sql
/* 2. TAHSİLAT KAYDINI OLUŞTUR (29.12.2025) */
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 6, 6211.00, '2025-12-29', '2025-12-29', 'Yakup Akyüz (KT1 D6) - Ekim-
Kasım-Aralık Tüm Borçlar Kapatıldı');


/* 1. KASIM YAKIT BORCUNU TAMAMEN KAPAT (En eski borç) */
UPDATE debt_item
SET status = 'PAID'
WHERE unit_id = 30
  AND type = 'FUEL'
  AND period_month = '2025-11-01';

/* 2. ARALIK YAKIT BORCUNU 30 TL OLARAK GÜNCELLE */
-- Mevcut borcun sadece 30 TL'lik kısmını ödenmemiş olarak bırakıyoruz
UPDATE debt_item
SET expected_amount = 30.00
WHERE unit_id = 30
  AND type = 'FUEL'
  AND period_month = '2025-12-01';

/* 1. OCAK AİDATINI (800 TL) 'ÖDENDİ' OLARAK KAPAT */
UPDATE debt_item
SET status = 'PAID'
WHERE unit_id = 22
  AND type = 'DUES'
  AND period_month = '2026-01-01';

/* 2. ARALIK YAKITINI 35 TL KALACAK ŞEKİLDE GÜNCELLE */
-- 1172 TL olan borcun 1137 TL'si ödendi, 35 TL bakiye kaldı.
UPDATE debt_item
SET expected_amount = 35.00
WHERE unit_id = 22
  AND type = 'FUEL'
  AND period_month = '2025-12-01';

/* 3. TAHSİLAT KAYDINI OLUŞTUR (Toplam 1937 TL) */
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 22, 1937.00, CURRENT_DATE, CURRENT_DATE, 'Kamil Önay - Ocak Aidat (Tam)
ve Aralık Yakıt (1137 TL ödendi, 35 TL kaldı)');

/* 1. ARALIK YAKIT BORCUNU 35 TL BAKİYE KALACAK ŞEKİLDE GÜNCELLE */
UPDATE debt_item
SET expected_amount = 35.00
WHERE unit_id = 29
  AND type = 'FUEL'
  AND period_month = '2025-12-01';
```

```sql
/* 2. TAHSİLAT KAYDINI GİR (861 TL) */
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 29, 861.00, CURRENT_DATE, CURRENT_DATE, 'Emrah Durkan - Aralık Yakıt
(861 TL ödendi, 35 TL kaldı)');

/* 1. OCAK AİDATINI 'ÖDENDİ' OLARAK KAPAT */
UPDATE debt_item
SET status = 'PAID'
WHERE unit_id = 14
  AND type = 'DUES'
  AND period_month = '2026-01-01';

/* 2. TAHSİLAT KAYDINI GİR (800 TL) */
INSERT INTO payment (complex_id, unit_id, amount, created_at, process_date,
description)
VALUES (1, 14, 800.00, CURRENT_DATE, CURRENT_DATE, 'Fikret Demirel - Ocak Aidat
Ödemesi');
```

[SQL Code for Legacy Data Migration Omitted - Corresponds to source 785-1261]

3.6. Database Validation and Test Queries

Test scenarios prepared to test the accuracy of the system, verify financial reports, and measure the performance of the SQL architecture are as follows:

- **Unit Count Check:** Should be 56.
- **Building Count Check:** Should be 7.
- **Site Count Check:** Should be 1.

```sql
-- Daire (UNIT) Sayısı Kontrolü: 56 olmalı.
SELECT COUNT(*) FROM unit;

-- Blok (BUILDING) Sayısı Kontrolü: 7 olmalı.
SELECT COUNT(*) FROM building;

-- Site (COMPLEX_PROPERTIES) Sayısı Kontrolü: 1 olmalı.
SELECT COUNT(*) FROM complex_properties;


SELECT id, username, role FROM "user";
-- Çıktıda 'muhasebe_yoneticisi' ve 'MANAGER' rolünü görmelisiniz.

SELECT
    u.unit_number,              -- Daire numarası
    u.owner_name,               -- Ev sahibi
    b.name AS building_name,    -- Blok adı
```

```
    ut.name AS unit_type        -- Daire tipi
FROM unit u
JOIN building b
    ON u.building_id = b.id
JOIN unit_type ut
    ON u.unit_type_id = ut.id
JOIN complex_properties cp
    ON b.complex_id = cp.id
WHERE cp.id = 1                -- İlgili site (örnek: complex_id = 1)
ORDER BY u.id
LIMIT 5;
```

[SQL Code for Test Queries Omitted - Corresponds to source 1269-1289]

CHAPTER 4: APPLICATION LOGIC AND MAIN SOURCE CODES

Following the database layer, this section details the Python logic that controls the operation of the application.

4.1. Application Architecture and Functional Details

`main.py`, the heart of the application, coordinates both backend (database communication) and frontend (Streamlit interface) processes with a code block of approximately 900 lines. This section discusses the modular structure of the application and the technical details of its critical functions.

4.1.1. Security and Environment Variables (.env & venv)

"Secure Software Development" principles were strictly adhered to during the development of the project:

- **Virtual Environment (venv):** The application runs in an isolated `venv` environment to avoid conflicts with system-wide libraries. This ensures that the project's dependencies (listed in `requirements.txt`) remain specific only to this project.
- **Sensitive Data Management (.env):** Database credentials, host information, and port numbers are stored in an external `.env` file instead of being hardcoded. These data points are read at runtime using the `python-dotenv` library, ensuring source code security and preventing sensitive information from being exposed on platforms like GitHub.
- **Authentication:** A simple yet effective login panel is implemented using the `check_password()` function. Once credentials are verified, session management is handled via `st.session_state` to maintain the user's state across the application.

4.1.2. Critical Backend Functions

The core functions responsible for automating database operations and maintaining financial logic are as follows:

- **`get_connection()`:** This is the primary bridge establishing the link between the application and the database. It utilizes `try-except` blocks to catch connection errors, ensuring the application remains stable and prevents system crashes.
- **`tahsilat_kaydet()` & `kaydet_odeme()`:** These functions implement the project’s most complex algorithm. When a payment is

processed, the system does not simply execute an `INSERT` operation; it dynamically identifies the specific unit's oldest outstanding debts (debt_items) and iterates through them, updating their status to "Paid" until the payment amount is fully utilized (**FIFO Logic**).

- **`toplu_yakit_yukle():`** This function utilizes the **Pandas** library to parse uploaded CSV files row by row. It performs bulk debt creation by mapping unit numbers to their corresponding unique database IDs, ensuring data integrity during mass updates.
- **`get_genel_istatistikler():`** This function performs real-time financial reporting. It utilizes SQL `SUM` and `GROUP BY`aggregations to calculate the current cash balance, total receivables, and total expenses instantly.

4.1.3. User Interface Components (Streamlit UI)

The interface design is focused on presenting complex financial data through a simplified, intuitive experience for the end-user (property manager):

- **Unit Detail Dialog Screen (`@st.dialog`):** When a specific unit is selected, this modal window opens to display debt statements and payment history using a tabbed structure (`st.tabs`). This modular approach keeps the main dashboard clean and organized.
- **Metric Cards (`st.metric`):** Critical performance indicators such as the "Cash Balance" and "Total Receivables" are visualized through colorful metric cards. These cards utilize color coding—red for deficits and green for positive balances—to provide immediate visual feedback.
- **Data Exporting:** Integrated via the `st.download_button` component, this feature allows managers to export the current debt list into a UTF-8 encoded CSV file in seconds, facilitating external reporting and archiving.

4.2. Main Source Codes (main.py)

The complete source code, which incorporates all the functions, database connections, and user interface logic discussed in the previous sections, is provided below:

```
import streamlit as st
import psycopg2
import pandas as pd
import os
from dotenv import load_dotenv
```

```python
load_dotenv()


def get_connection():
    try:
        conn = psycopg2.connect(
            host=os.getenv("DB_HOST"),
            database=os.getenv("DB_NAME"),
            user=os.getenv("DB_USER"),
            password=os.getenv("DB_PASS"),
            port=os.getenv("DB_PORT")
        )
        return conn
    except Exception as e:
        st.error(f"Veritabanı bağlantı hatası: {e}")
        return None

def check_password():
    """Kullanıcı adı ve şifreyi kontrol eder."""
    if "password_correct" not in st.session_state:
        st.session_state["password_correct"] = False

    if st.session_state["password_correct"]:
        return True

    # Giriş Ekranı
    st.title("🔐 Yönetim Paneli")
    with st.container():
        user = st.text_input("Kullanıcı Adı")
        pw = st.text_input("Şifre", type="password")
        if st.button("Giriş Yap"):
            if user == "nidakd" and pw == "muhasebe123":
                st.session_state["password_correct"] = True
                st.rerun() # Bilgiler doğruysa sayfayı yenile ve içeri al
            else:
                st.error("❌ Kullanıcı adı veya şifre hatalı!")
    return False


def toplu_yakit_yukle(data_df, donem):
    conn = get_connection()
    if conn:
        try:
            cur = conn.cursor()
            for index, row in data_df.iterrows():
                # Daire numarasına göre unit_id'yi buluyoruz
                cur.execute("SELECT id FROM unit WHERE unit_number = %s",
(str(row['Daire No']),))
                unit_res = cur.fetchone()
                if unit_res:
```

```
                    unit_id = unit_res[0]
                    tutar = row['Tutar']
                    cur.execute("""
                        INSERT INTO debt_item (unit_id, type, expected_amount,
period_month, status)
                        VALUES (%s, 'FUEL', %s, %s, 'UNPAID')
                    """, (unit_id, tutar, donem))
            conn.commit()
            cur.close()
            conn.close()
            return True
        except Exception as e:
            st.error(f"Yükleme hatası: {e}")
            return False


def tahsilat_kaydet(unit_id, amount, p_type, description):
    conn = get_connection()
    if conn:
        try:
            cur = conn.cursor()
            # 1. Ödemeyi kaydet
            cur.execute("""
                INSERT INTO payment (complex_id, unit_id, amount, payment_type,
process_date, description)
                VALUES (%s, %s, %s, %s, CURRENT_TIMESTAMP, %s) RETURNING id
            """, (st.session_state.selected_site_id, unit_id, amount, p_type,
description))

            # 2. BORÇ KAPATMA MANTIĞI:
            # Dairenin ödenmemiş en eski borçlarını getir
            cur.execute("""
                SELECT id, expected_amount FROM debt_item
                WHERE unit_id = %s AND status != 'PAID'
                ORDER BY period_month ASC
            """, (unit_id,))

            borclar = cur.fetchall()
            kalan_odeme = float(amount)

            for borc_id, borc_tutar in borclar:
                if kalan_odeme >= float(borc_tutar):
                    # Borcun tamamını kapat
                    cur.execute("UPDATE debt_item SET status = 'PAID' WHERE id
= %s", (borc_id,))
                    kalan_odeme -= float(borc_tutar)
                elif kalan_odeme > 0:
                    pass

            conn.commit()
            cur.close()
```

```
            conn.close()
            return True
        except Exception as e:
            st.error(f"Hata: {e}")
    return False

def get_aylik_tahsilat_verisi(site_id):
    conn = get_connection()
    if conn:
        try:
            # Son 6 ayın tahsilatlarını getirir
            query = """
                SELECT TO_CHAR(process_date, 'Mon') as ay, SUM(amount) as toplam,
MIN(process_date) as sirala
                FROM payment
                WHERE complex_id = %s
                GROUP BY ay
                ORDER BY sirala ASC
            """
            df = pd.read_sql(query, conn, params=(site_id,))
            conn.close()
            return df
        except: return pd.DataFrame()
    return pd.DataFrame()

def get_blok_borc_verisi(site_id):
    conn = get_connection()
    if conn:
        try:
            # Bloklara göre toplam borç dağılımı
            query = """
                SELECT b.name as "Blok", SUM(d.expected_amount) as "Toplam Borç"
                FROM debt_item d
                JOIN unit u ON d.unit_id = u.id
                JOIN building b ON u.building_id = b.id
                WHERE b.complex_id = %s AND d.status != 'PAID'
                GROUP BY b.name
                ORDER BY "Toplam Borç" DESC
            """
            df = pd.read_sql(query, conn, params=(site_id,))
            conn.close()
            return df
        except: return pd.DataFrame()
    return pd.DataFrame()

def get_genel_istatistikler(site_id):
    conn = get_connection()
    stats = {"toplam_alacak": 0.0, "toplam_tahsilat": 0.0, "kasa_mevcut": 0.0}
    if conn:
        try:
            cur = conn.cursor()
```

```
            # 1. Alacaklar
            cur.execute("SELECT SUM(expected_amount) FROM debt_item WHERE status !=
'PAID'")
            stats["toplam_alacak"] = float(cur.fetchone()[0] or 0)

            # 2. Gelirler
            cur.execute("SELECT SUM(amount) FROM payment WHERE complex_id = %s",
(site_id,))
            toplam_gelir = float(cur.fetchone()[0] or 0)
            stats["toplam_tahsilat"] = toplam_gelir

            # 3. Giderler
            cur.execute("SELECT SUM(amount) FROM account_transaction WHERE type =
'EXPENSE'")
            toplam_gider = float(cur.fetchone()[0] or 0)

            stats["kasa_mevcut"] = toplam_gelir - toplam_gider

            cur.close()
            conn.close()
        except Exception as e:
            print(f"İstatistik hatası: {e}")
    return stats


def kaydet_personel_odeme(site_id, personel_id, miktar, aciklama):
    conn = get_connection()
    if conn:
        try:
            cur = conn.cursor()
            # 1. Personel ödemesini kaydet
            cur.execute("""
                INSERT INTO account_transaction (complex_id, type, category,
amount, process_date, description)
                VALUES (%s, 'EXPENSE', 'Personel Maaş', %s, CURRENT_TIMESTAMP, %s)
            """, (site_id, miktar, aciklama))

            conn.commit()
            cur.close()
            conn.close()
            return True
        except Exception as e:
            st.error(f"Ödeme kaydedilirken hata: {e}")
            return False


def get_detayli_borc(daire_id):
    conn = get_connection()
    detay = {"aidat": 0.0, "yakit": 0.0, "toplam": 0.0}
    if conn:
        try:
```

```
            cur = conn.cursor()
            # Aidat ve Yakıtı ayrı ayrı topluyoruz
            query = """
                SELECT type, SUM(expected_amount)
                FROM debt_item
                WHERE unit_id = %s AND status != 'PAID'
                GROUP BY type
            """
            cur.execute(query, (daire_id,))
            rows = cur.fetchall()
            for row in rows:
                if row[0] == 'DUES': detay["aidat"] = float(row[1])
                if row[0] == 'FUEL': detay["yakit"] = float(row[1])

            detay["toplam"] = detay["aidat"] + detay["yakit"]
            cur.close()
            conn.close()
        except:
            pass
    return detay

def kaydet_gider(site_id, miktar, kategori, aciklama):
    conn = get_connection()
    if conn:
        try:
            cur = conn.cursor()
            # 'transaction_type' yerine 'type'
            query = """
                INSERT INTO account_transaction (complex_id, type, category,
amount, process_date, description)
                VALUES (%s, 'EXPENSE', %s, %s, CURRENT_TIMESTAMP, %s)
            """
            cur.execute(query, (site_id, kategori, miktar, aciklama))
            conn.commit()
            cur.close()
            conn.close()
            return True
        except Exception as e:
            st.error(f"Gider kaydedilirken hata oluştu: {e}")
            return False

def get_daire_odemeleri(daire_id):
    conn = get_connection()
    if conn:
        try:
            # Dairenin yaptığı tüm ödemeleri en yeni tarihten başlayarak
getiriyoruz
            query = """
                SELECT process_date, amount, description
                FROM payment
                WHERE unit_id = %s
```

```
                ORDER BY process_date DESC
            """
            df = pd.read_sql(query, conn, params=(daire_id,))
            conn.close()

            if not df.empty:
                # Tarihi gg/aa/yyyy formatına çevirelim
                df['process_date'] =
pd.to_datetime(df['process_date']).dt.strftime('%d/%m/%Y')
                # Sütunları Türkçeleştirelim
                df.columns = ['Ödeme Tarihi', 'Tutar (TL)', 'Açıklama']
            return df
        except:
            return pd.DataFrame()
    return pd.DataFrame()

def get_daire_extresi(daire_id):
    conn = get_connection()
    if conn:
        try:
            query = """
                SELECT period_month, type, expected_amount
                FROM debt_item
                WHERE unit_id = %s AND status != 'PAID'
                ORDER BY period_month ASC
            """
            df = pd.read_sql(query, conn, params=(daire_id,))
            conn.close()

            if not df.empty:
                # Tarihleri Türkçeleştirme
                df['period_month'] =
pd.to_datetime(df['period_month']).dt.strftime('%m/%Y')
                # Türleri Türkçeleştirme
                df['type'] = df['type'].replace({'DUES': '🏠 Aidat', 'FUEL': '🔥
Yakıt'})
                # Sütun isimlerini Türkçeleştirme
                df.columns = ['Dönem', 'Borç Türü', 'Tutar (TL)']
            return df
        except:
            return pd.DataFrame()
    return pd.DataFrame()

@st.dialog("Daire Cari Hesap Detayı", width="large")
def daire_detay_penceresi(row, borclar):
    st.write(f"### 🚪 Daire No: {row['unit_number']} - {row['owner_name']}")

    # Üst Bilgi Kartları
    c1, c2, c3 = st.columns(3)
    c1.metric("Güncel Borç", f"₺{borclar['toplam']:,.2f}")
```

```
    # Sekmeli yapı (Tabs) kullanarak ekranı daha düzenli yapalım
    tab1, tab2 = st.tabs(["📉 Ödenmemiş Borçlar", "💰 Ödeme Geçmişi (Makbuzlar)"])

    with tab1:
        st.write("#### Dönem Bazlı Borç Dökümü")
        extre_df = get_daire_extresi(int(row['id']))
        if not extre_df.empty:
            st.dataframe(extre_df, use_container_width=True, hide_index=True)
        else:
            st.success("Harika! Bu dairenin hiç borcu yok.")

    with tab2:
        st.write("#### Yapılan Tahsilatlar")
        odeme_df = get_daire_odemeleri(int(row['id']))
        if not odeme_df.empty:
            st.dataframe(odeme_df, use_container_width=True, hide_index=True)
            st.info("💡 Yukarıdaki liste, sistemde kayıtlı olan tüm banka ve nakit
ödemelerini gösterir.")
        else:
            st.warning("Bu daireye ait henüz bir ödeme kaydı bulunamadı.")

    st.divider()
    if st.button("Kapat", use_container_width=True):
        st.rerun()

# Ödemeyi veritabanına kaydeden fonksiyon
def kaydet_odeme(site_id, daire_id, tutar, aciklama):
    conn = get_connection()
    if conn:
        try:
            cur = conn.cursor()
            # 1. Ödemeyi Kaydet
            cur.execute("""
                INSERT INTO payment (complex_id, unit_id, amount, process_date,
description)
                VALUES (%s, %s, %s, CURRENT_TIMESTAMP, %s)
            """, (site_id, daire_id, tutar, aciklama))

            # 2. BORÇ KAPATMA MANTIĞI:
            # Dairenin ödenmemiş en eski borçlarını tek tek bulalım
            cur.execute("""
                SELECT id, expected_amount FROM debt_item
                WHERE unit_id = %s AND status != 'PAID'
                ORDER BY period_month ASC
            """, (daire_id,))

            borclar = cur.fetchall()
            kalan_para = float(tutar)

            for borc_id, borc_tutar in borclar:
                if kalan_para >= float(borc_tutar):
```

```
                    # Borcun tamamını ödeyecek kadar para var
                    cur.execute("UPDATE debt_item SET status = 'PAID' WHERE id
= %s", (borc_id,))
                    kalan_para -= float(borc_tutar)
                else:
                    # Para bitti, diğer borçlar ödenmemiş kalmaya devam eder
                    break

            conn.commit()
            cur.close()
            conn.close()
            return True
        except Exception as e:
            st.error(f"Kayıt hatası: {e}")
            return False
    return False


# Sayfa Genişlik Ayarı
st.set_page_config(page_title="Site Yönetim Paneli", layout="wide")

# --- SİTE SEÇİM MANTIĞI ---
if 'selected_site_id' not in st.session_state:
    # Karşılama Ekranı
    st.title("🏙️ Site Yönetim Sistemine Hoş Geldiniz")
    st.info("Devam etmek için lütfen yönetmek istediğiniz siteyi seçin.")

    conn = get_connection()
    if conn:
        query = "SELECT id, name FROM complex_properties"
        sites_df = pd.read_sql(query, conn)
        conn.close()

        if not sites_df.empty:
            secilen_ad = st.selectbox("Site Seçiniz:", sites_df['name'])
            site_id = int(sites_df[sites_df['name'] == secilen_ad]['id'].values[0])

            if st.button("Sisteme Giriş Yap"):
                st.session_state.selected_site_id = site_id
                st.session_state.selected_site_name = secilen_ad
                st.rerun()
else:

    if not check_password():
        st.stop()


    # --- ANA PANEL (SİTE SEÇİLDİKTEN SONRA) ---
    st.sidebar.title(f"📍 {st.session_state.selected_site_name}")
    st.sidebar.success(f"🔓 Giriş: nidakd")
```

```python
    # Sol Menü Modülleri
    menu = st.sidebar.radio(
    "Menü",
    ["📊 Genel Bakış", "💰 Kasa (Tahsilat)", "🏢 Bloklar ve Daireler", "📉 Giderler",
"👷 Personeller", "🚜 Toplu İşlemler"]
)

    if st.sidebar.button("🔄 Site Değiştir"):
        del st.session_state.selected_site_id
        st.session_state["password_correct"] = False
        st.rerun()


    if menu == "📊 Genel Bakış":
        st.header(f"📊 {st.session_state.selected_site_name} Genel Durum")

        # Fonksiyonu çağırıp istatistikleri alıyoruz
        stats = get_genel_istatistikler(st.session_state.selected_site_id)

        # Üst Metrik Kartları
        m1, m2, m3 = st.columns(3)
        with m1:
            st.metric("💰 Toplam Tahsilat (Kasa)",
f"₺{stats['toplam_tahsilat']:,.2f}")
        with m2:
            st.metric("🚨 Bekleyen Toplam Alacak",
f"₺{stats['toplam_alacak']:,.2f}")
        with m3:
            st.metric("🏦 Kasa Bakiyesi", f"₺{stats['kasa_mevcut']:,.2f}")

        st.divider()

        # --- RAPORLAMA VE EXCEL ÇIKTISI ---
        st.subheader("📩 Veri Dışarı Aktar (Excel/CSV)")

        if st.button("📊 Güncel Borç Listesini Hazırla"):
            conn = get_connection()
            sorgu = """
                SELECT b.name as "Blok",
                       u.unit_number as "Daire No",
                       u.owner_name as "Ev Sahibi",
                       d.type as "Tur",
                       d.expected_amount as "Tutar",
                       d.period_month as "Donem",
                       d.status as "Durum"
                FROM debt_item d
                JOIN unit u ON d.unit_id = u.id
                JOIN building b ON u.building_id = b.id
                ORDER BY b.name ASC, u.unit_number::int ASC, d.period_month DESC
            """
            df_indir = pd.read_sql(sorgu, conn)
```

```
            conn.close()

            if not df_indir.empty:
                df_indir['Tur'] = df_indir['Tur'].replace({'DUES': 'Aidat', 'FUEL':
'Yakıt'})
                df_indir['Durum'] = df_indir['Durum'].replace({'UNPAID':
'Ödenmedi', 'PAID': 'Ödendi'})
                df_indir['Donem'] =
pd.to_datetime(df_indir['Donem']).dt.strftime('%m/%Y')
                df_indir.columns = ["Blok", "Daire No", "Ev Sahibi", "Borç Türü",
"Tutar (TL)", "Dönem", "Ödeme Durumu"]
                csv = df_indir.to_csv(index=False).encode('utf-8-sig')

                st.success("Liste başarıyla hazırlandı!")
                st.download_button(
                    label="📩 Borç Listesini İndir (Türkçe)",
                    data=csv,
                    file_name='site_borc_listesi_guncel.csv',
                    mime='text/csv',
                    use_container_width=True
                )
            else:
                st.info("İndirilecek borç verisi bulunamadı.")

        st.divider()

        # --- DİNAMİK GRAFİKLER ---
        c1, c2 = st.columns(2)

        with c1:
            st.subheader("📉 Aylık Tahsilat Trendi")
            tahsilat_df =
get_aylik_tahsilat_verisi(st.session_state.selected_site_id)
            if not tahsilat_df.empty:
                st.bar_chart(tahsilat_df, x='ay', y='toplam', color="#2E7D32")
            else:
                st.info("Henüz tahsilat verisi bulunmuyor.")

        with c2:
            st.subheader("🏢 Blok Bazlı Borç Dağılımı")
            blok_borc_df = get_blok_borc_verisi(st.session_state.selected_site_id)
            if not blok_borc_df.empty:
                st.bar_chart(blok_borc_df, x='Blok', y='Toplam Borç',
color="#C62828")
            else:
                st.success("✅ Tüm blokların borcu ödenmiş!")

        st.divider()

        # --- ÖZET TABLO ---
        st.subheader("📂 Genel Borç Dağılımı")
```

```
        conn_genel = get_connection()
        if conn_genel:
            dist_query = "SELECT type, SUM(expected_amount) FROM debt_item WHERE
status != 'PAID' GROUP BY type"
            dist_df = pd.read_sql(dist_query, conn_genel)
            conn_genel.close()

            if not dist_df.empty:
                dist_df['type'] = dist_df['type'].replace({'DUES': 'Aidat', 'FUEL':
'Yakıt'})
                st.dataframe(dist_df, use_container_width=True, hide_index=True)


    elif menu == "🏢 Bloklar ve Daireler":
        st.header("🏢 Blok Bazlı Borç Takip Paneli")


        conn = get_connection()

        # 1. Blokları çekelim
        bloklar_query = f"SELECT id, name FROM building WHERE complex_id =
{st.session_state.selected_site_id}"
        bloklar_df = pd.read_sql(bloklar_query, conn)

        # 2. Üstten blok seçimi yapalım
        secilen_blok_adi = st.selectbox("İncelemek istediğiniz bloğu seçin:",
bloklar_df['name'])
        secilen_blok_id = bloklar_df[bloklar_df['name'] ==
secilen_blok_adi]['id'].values[0]

        # 3. Seçilen bloğun dairelerini getirelim
        conn = get_connection()
        daire_query = f"SELECT id, unit_number, owner_name FROM unit WHERE
building_id = {int(secilen_blok_id)} ORDER BY unit_number::int"
        daireler = pd.read_sql(daire_query, conn)
        conn.close()

        # 4. Görsel Grid (Kutucuklar) Oluşturma
        st.write(f"### {secilen_blok_adi} Bloğu Daire Durumları")

        # Her satırda 4 daire olacak şekilde kolonlar
        cols = st.columns(4)

        for index, row in daireler.iterrows():
            borclar = get_detayli_borc(int(row['id']))
            col_index = index % 4

            with cols[col_index]:
                with st.container(border=True):
                    st.markdown(f"### 🚪 Daire {row['unit_number']}")
```

```
                    st.caption(f"👤 {row['owner_name']}")

                    # Borç durumuna göre renkli gösterge
                    if borclar["toplam"] > 0:
                        st.error(f"Güncel Borç: ₺{borclar['toplam']:,.2f}")
                    else:
                        st.success("✅ Borç bulunmuyor")

                    if st.button(f"🔍 Detayları Gör", key=f"btn_{row['id']}",
use_container_width=True):
                        daire_detay_penceresi(row, borclar)

    elif menu == "💰 Kasa (Tahsilat)":
        st.header("💰 Yeni Tahsilat Girişi")

        conn = get_connection()


        # 1. Daire listesini çekelim (Seçim kutusu için)
        daire_sorgu = f"""
            SELECT u.id, b.name || ' - Daire ' || u.unit_number || ' (' ||
u.owner_name || ')' as label
            FROM unit u
            JOIN building b ON u.building_id = b.id
            WHERE b.complex_id = {st.session_state.selected_site_id}
        """
        daireler_df = pd.read_sql(daire_sorgu, conn)
        conn.close()

        # 2. Ödeme Formu
        with st.form("tahsilat_formu"):
            secilen_daire_label = st.selectbox("Ödeme Yapan Daire:",
daireler_df['label'])
            daire_id = int(daireler_df[daireler_df['label'] ==
secilen_daire_label]['id'].values[0])

            tutar = st.number_input("Ödenen Tutar (TL):", min_value=0.0,
step=100.0)
            aciklama = st.text_input("Açıklama:", placeholder="Örn: Ocak 2026
Aidatı")

            submit_button = st.form_submit_button("💳 Ödemeyi Sisteme İşle")

            if submit_button:
                if tutar > 0:
                    # Tüm sayısal değerleri int() veya float() içine alarak
garantiye alıyoruz
                    s_id = int(st.session_state.selected_site_id)
                    d_id = int(daire_id)
                    t_val = float(tutar)
```

```
                    basarili = kaydet_odeme(s_id, d_id, t_val, aciklama)
                    if basarili:
                        st.success(f"Başarılı! {secilen_daire_label} için {t_val}
TL ödeme alındı.")
                        st.balloons()


    elif menu == "👷 Personeller":
        st.header("👷 Personel Yönetimi ve Maaş Takibi")

        tab1, tab2 = st.tabs(["👥 Personel Listesi", "💸 Maaş/Ödeme Yap"])

        with tab1:
            st.subheader("Aktif Personeller")
            conn = get_connection()
            # Veritabanındaki personelleri çekiyoruz
            personel_df = pd.read_sql(f"SELECT name as \"İsim\", role as \"Görev\",
salary as \"Maaş\" FROM employee WHERE complex_id =
{st.session_state.selected_site_id}", conn)
            conn.close()

            if not personel_df.empty:
                st.table(personel_df)
            else:
                st.info("Henüz sisteme kayıtlı personel bulunmuyor.")

        with tab2:
            st.subheader("Ödeme Formu")
            with st.form("personel_odeme_formu", clear_on_submit=True):
                # Personel seçimi için listeyi tekrar çekelim
                conn = get_connection()
                p_list = pd.read_sql(f"SELECT id, name FROM employee WHERE
complex_id = {st.session_state.selected_site_id}", conn)
                conn.close()

                if not p_list.empty:
                    secilen_p = st.selectbox("Personel Seçin:", p_list['name'])
                    p_id = p_list[p_list['name'] == secilen_p]['id'].values[0]

                    tutar = st.number_input("Ödeme Tutarı (TL):", min_value=0.0,
step=100.0)
                    aciklama = st.text_input("Açıklama:", value=f"{secilen_p} -
Ocak 2026 Maaş Ödemesi")

                    if st.form_submit_button("Ödemeyi Onayla"):
                        if tutar > 0:
                            if
kaydet_personel_odeme(st.session_state.selected_site_id, int(p_id), tutar,
aciklama):
                                st.success(f"{secilen_p} personeline ₺{tutar} ödeme
yapıldı ve kasadan düşüldü.")
```

```
                        else:
                            st.warning("Lütfen tutar giriniz.")
                else:
                    st.error("Ödeme yapabilmek için önce personel eklemelisiniz.")



    elif menu == "📒 Giderler":
        st.header("💸 Site Gider Yönetimi")

        tab1, tab2 = st.tabs(["➕ Yeni Gider Ekle", "📋 Geçmiş Giderler"])

        with tab1:
            with st.form("gider_formu", clear_on_submit=True):
                col1, col2 = st.columns(2)
                with col1:
                    miktar = st.number_input("Harcama Tutarı (TL):", min_value=0.0,
step=50.0)
                    kategori = st.selectbox("Gider Kategorisi:",
                        ["Temizlik Ücreti", "Asansör Bakımı", "Elektrik Faturası",
"Yakıt Ödemesi", "Su Faturası", "Huzur Hakkı", "Temizlik Malzemesi", "Diğer"])
                with col2:
                    tarih = st.date_input("Harcama Tarihi:")
                    aciklama = st.text_area("Harcama Detayı:", placeholder="Örn: X
asansör firması aylık bakım bedeli")

                submit = st.form_submit_button("❌ Gideri Kasadan Düş")

                if submit:
                    if miktar > 0:
                        if kaydet_gider(st.session_state.selected_site_id, miktar,
kategori, aciklama):
                            st.success(f"Başarılı: {kategori} için ₺{miktar} gider
kaydedildi.")
                            st.balloons()
                    else:
                        st.warning("Lütfen harcama tutarını giriniz.")

        with tab2:
            st.subheader("📋 Son Harcamalar")
            conn = get_connection()
            # account_transaction tablosundaki EXPENSE (Gider) kayıtlarını
çekiyoruz
            gider_query = """
                    SELECT process_date as "Tarih", category as "Kategori", amount
as "Tutar", description as "Açıklama"
                    FROM account_transaction
                    WHERE type = 'EXPENSE'
                    ORDER BY process_date DESC
            """
            giderler_df = pd.read_sql(gider_query, conn)
```

```
            conn.close()

            if not giderler_df.empty:
                st.dataframe(giderler_df, use_container_width=True,
hide_index=True)
            else:
                st.info("Henüz kaydedilmiş bir gider bulunmuyor.")

    elif menu == "📢 Toplu İşlemler":
        st.header("📢 Toplu Borçlandırma Paneli")
        tab1, tab2, tab3 = st.tabs(["🏠 Sabit Aidat", "🔥 Yakıt (Excel)", "📅 Geçmiş
Borç Yükle"])

        with tab1:
            st.subheader("Tüm Siteye Sabit Aidat Yansıt")
            st.info("💡 Bu işlem, muaf olarak işaretlenen yöneticileri otomatik
olarak atlar.")

            with st.form("toplu_aidat_form"):
                col1, col2 = st.columns(2)
                with col1:
                    aidat_tutar = st.number_input("Aidat Tutarı (TL):",
min_value=0.0, step=50.0, value=500.0)
                with col2:
                    aidat_ay = st.date_input("Aidat Dönemi:", key="aidat_date")

                if st.form_submit_button("🚀 Aidatları Tüm Siteye Yansıt"):
                    conn = get_connection()
                    if conn:
                        try:
                            cur = conn.cursor()
                            # --- unit tablosunda complex_id olmadığı için JOIN
kullanıyoruz ---
                            cur.execute("""
                                SELECT u.id
                                FROM unit u
                                JOIN building b ON u.building_id = b.id
                                WHERE b.complex_id = %s
                                AND (u.is_exempt IS DISTINCT FROM TRUE)
                            """, (st.session_state.selected_site_id,))

                            daireler = cur.fetchall()

                            for d in daireler:
                                cur.execute("""
                                    INSERT INTO debt_item (unit_id, type,
expected_amount, period_month, status)
                                    VALUES (%s, 'DUES', %s, %s, 'UNPAID')
                                """, (d[0], aidat_tutar, aidat_ay))

                            conn.commit()
```

```
                            cur.close()
                            conn.close()
                            st.success(f"Başarılı! {len(daireler)} daireye
₺{aidat_tutar} aidat borcu girildi.")
                            st.balloons()
                        except Exception as e:
                            st.error(f"Hata: {e}")

        with tab2:
            st.subheader("Daire Bazlı Farklı Yakıt Girişi")
            st.info("💡 Şablondaki 'Dönem' kısmını YYYY-AA-GG (Örn: 2026-01-01)
formatında doldurun.")

            # 1. ADIM: Şablon Hazırlama (Dönem sütunu eklendi)
            conn = get_connection()
            sablon_sorgu = """
                SELECT u.unit_number as "Daire No", u.owner_name as "Ev Sahibi",
                       0.0 as "Tutar", '2026-01-01' as "Donem"
                FROM unit u
                JOIN building b ON u.building_id = b.id
                WHERE b.complex_id = %s
                ORDER BY b.name ASC, u.unit_number::int ASC
            """
            sablon_df = pd.read_sql(sablon_sorgu, conn,
params=(st.session_state.selected_site_id,))
            conn.close()

            csv_sablon = sablon_df.to_csv(index=False).encode('utf-8-sig')

            st.download_button(
                label="📥 1. Adım: Yakıt Şablonunu İndir",
                data=csv_sablon,
                file_name='yakit_yukleme_sablonu.csv',
                mime='text/csv'
            )

            st.divider()

            # 2. ADIM: Dosya Yükleme
            yuklenen_dosya = st.file_uploader("Doldurduğunuz dosyayı yükleyin",
type=['csv'], key="yakit_csv_up")

            if yuklenen_dosya and st.button("🚀 3. Adım: Yakıt Borçlarını İşle"):
                try:
                    df_yuklenen = pd.read_csv(yuklenen_dosya)
                    conn = get_connection()
                    cur = conn.cursor()

                    basarili_sayisi = 0
                    for index, row in df_yuklenen.iterrows():
                        # Daire numarasına göre ID bul
```

```
                        cur.execute("""
                            SELECT u.id FROM unit u
                            JOIN building b ON u.building_id = b.id
                            WHERE u.unit_number = %s AND b.complex_id = %s
                        """, (str(row['Daire No']),
st.session_state.selected_site_id))
                        unit_res = cur.fetchone()

                        # Tutar 0'dan büyükse kaydet
                        if unit_res and float(row['Tutar']) > 0:
                            # Excel'deki 'Donem' sütununu kullanıyoruz
                            cur.execute("""
                                INSERT INTO debt_item (unit_id, type,
expected_amount, period_month, status)
                                VALUES (%s, 'FUEL', %s, %s, 'UNPAID')
                            """, (unit_res[0], row['Tutar'], row['Donem']))
                            basarili_sayisi += 1

                    conn.commit()
                    cur.close()
                    conn.close()
                    st.success(f"İşlem Tamam! {basarili_sayisi} daireye yakıt
borçları tarihleriyle birlikte girildi.")
                    st.balloons()
                except Exception as e:
                    st.error(f"Dosya işlenirken hata oluştu: {e}")

        with tab3:
            st.subheader("📅 Geçmiş Dönem Detaylı Borç Aktarımı")
            st.info("💡 Blok bazlı ayrım için lütfen yeni şablonu indirin ve
kullanın.")

            # 1. ADIM: Blok Bilgili Şablon Hazırlama
            conn = get_connection()
            sablon_sorgu = """
                SELECT b.name as "Blok", u.unit_number as "Daire No", u.owner_name
as "Ev Sahibi",
                       0.0 as "Gecmis_Aidat", 0.0 as "Ekim_Yakit",
                       0.0 as "Kasim_Yakit", 0.0 as "Aralik_Yakit", 0.0 as "Diger"
                FROM unit u
                JOIN building b ON u.building_id = b.id
                WHERE b.complex_id = %s
                ORDER BY b.name ASC, u.unit_number::int ASC
            """
            sablon_df = pd.read_sql(sablon_sorgu, conn,
params=(st.session_state.selected_site_id,))
            conn.close()

            sablon_df.columns = ["Blok", "Daire No", "Ev Sahibi", "Geçmiş Aidat
Borcu", "Ekim Yakıt", "Kasım Yakıt", "Aralık Yakıt", "Diğer Eksik Ödemeler"]
```

```
            st.download_button(
                label="📥 1. Adım: Yeni Bloklu Şablonu İndir",
                data=sablon_df.to_csv(index=False).encode('utf-8-sig'),
                file_name='bloklu_gecmis_borclar.csv',
                mime='text/csv'
            )

            st.divider()

            # 2. ADIM: Dosya Yükleme
            gecmis_dosya = st.file_uploader("Blok Bilgisi İçeren Listeyi Yükleyin",
type=['csv'], key="detayli_gecmis_up")

            if gecmis_dosya and st.button("🚀 2. Adım: Tüm Geçmişi Sisteme Aktar"):
                try:
                    ham_dosya = gecmis_dosya.getvalue().decode('utf-8-
sig').splitlines()
                    baslik_satiri_index = 0
                    for i, satir in enumerate(ham_dosya):
                        if "Blok" in satir or "Daire No" in satir:
                            baslik_satiri_index = i
                            break

                    gecmis_dosya.seek(0)
                    df_gecmis = pd.read_csv(gecmis_dosya, sep=None,
engine='python', encoding='utf-8-sig', skiprows=baslik_satiri_index)
                    df_gecmis.columns = [str(c).strip() for c in df_gecmis.columns]

                    conn = get_connection()
                    cur = conn.cursor()

                    # Önce mevcut yanlış yüklenen eski borçları temizleyelim
(Opsiyonel ama önerilir)
                    cur.execute("DELETE FROM debt_item WHERE period_month < '2026-
01-01' AND status = 'UNPAID'")

                    kayit_sayisi = 0
                    for index, row in df_gecmis.iterrows():
                        # Blok ve Daireyi beraber sorguluyoruz
                        blok_adi = str(row.get('Blok', '')).strip()
                        d_no = str(row.get('Daire No', '')).strip()

                        cur.execute("""
                            SELECT u.id FROM unit u
                            JOIN building b ON u.building_id = b.id
                            WHERE b.name = %s AND u.unit_number = %s AND
b.complex_id = %s
                        """, (blok_adi, d_no, st.session_state.selected_site_id))
                        unit_res = cur.fetchone()

                        if unit_res:
```

```
                            u_id = unit_res[0]
                            borc_kalemleri = [
                                (row.get('Geçmiş Aidat Borcu', 0), 'DUES', '2025-
12-31'),
                                (row.get('Ekim Yakıt', 0), 'FUEL', '2025-10-01'),
                                (row.get('Kasım Yakıt', 0), 'FUEL', '2025-11-01'),
                                (row.get('Aralık Yakıt', 0), 'FUEL', '2025-12-01'),
                                (row.get('Diğer Eksik Ödemeler', 0), 'OTHER',
'2025-12-30')
                            ]

                            for tutar, tur, tarih in borc_kalemleri:
                                try:
                                    tutar_val = float(str(tutar).replace(',', '.'))
if pd.notnull(tutar) else 0
                                except: tutar_val = 0

                                if tutar_val > 0:
                                    cur.execute("""
                                        INSERT INTO debt_item (unit_id, type,
expected_amount, period_month, status)
                                        VALUES (%s, %s, %s, %s, 'UNPAID')
                                        ON CONFLICT (unit_id, type, period_month)
DO UPDATE SET expected_amount = EXCLUDED.expected_amount
                                    """, (u_id, tur, tutar_val, tarih))
                                    kayit_sayisi += 1

                    conn.commit()
                    cur.close()
                    conn.close()
                    st.success(f"✅ Başarılı! {kayit_sayisi} adet kayıt doğru
bloklara eşlenerek yüklendi.")
                    st.balloons()
                except Exception as e:
                    st.error(f"❌ Hata: {e}")
```

[Python Source Code Omitted - Corresponds to source 1323-2089]

CHAPTER 5: APPLICATION WORKFLOW AND SCREENSHOTS (USER GUIDE)

This section provides a step-by-step visual explanation of the system's workflows and how the developed application is utilized by the end-user (Property Manager).

5.1. Secure Login and Dynamic Site Selection (Authentication)

The initial layer encountered upon launching the application is the security and configuration panel. This section consists of two primary stages:

1. **Site/Database Selection:** The user initiates the relevant database connection by selecting the specific apartment complex or site they intend to manage. This architecture demonstrates the **scalability** of the application, allowing the same infrastructure to serve multiple different site managements independently.
2. **Authentication (Login):** Following the selection phase, the `check_password()` function is invoked. This function tracks the user's session status using the `st.session_state` architecture. Access to the core functions of the application (Sidebar, Dashboard, etc.) is strictly restricted until the username and password credentials are successfully verified.

Technical Note: Once the login credentials are confirmed, the `st.rerun()` command is triggered, ensuring the interface reloads in "**Admin Panel**" mode. This process ensures that the application executes authorization protocols securely before granting access to sensitive financial data.

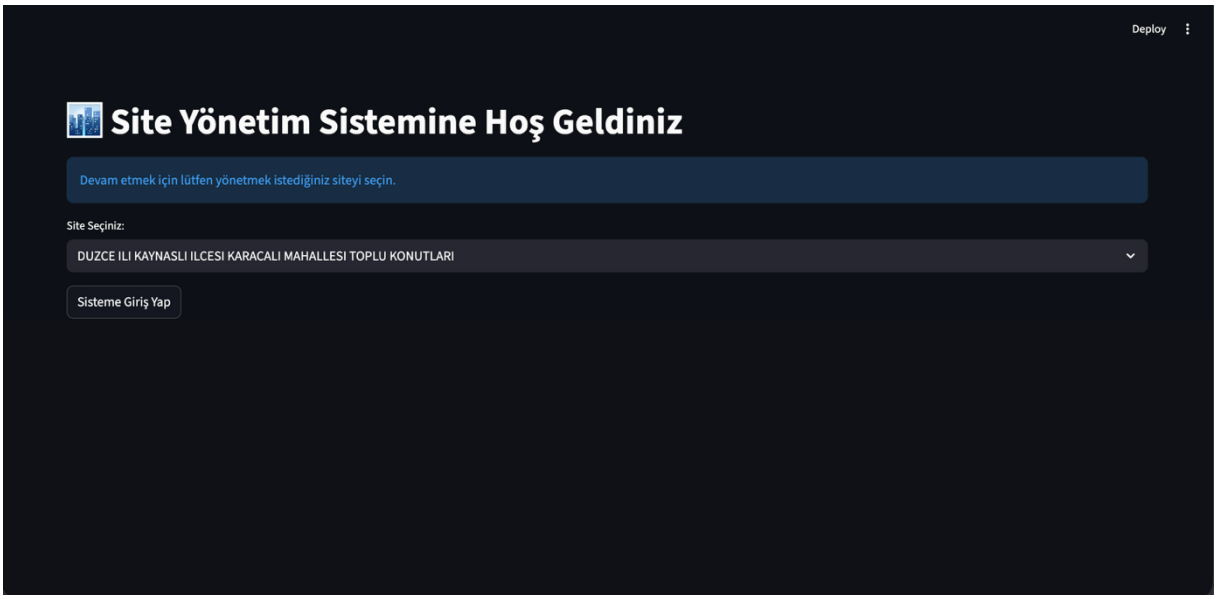


Figure 1 & 2: Application Login and Authentication Interface

5.2. Main Dashboard and Overview

Upon a successful login, the manager is presented with a comprehensive dashboard that summarizes the overall financial health of the site. By utilizing **Metric Cards** (`st.metric`), critical data points such as **Cash Balance**, **Total Receivables**, and **Total Expenses** are retrieved in real-time from the PostgreSQL database and displayed prominently.

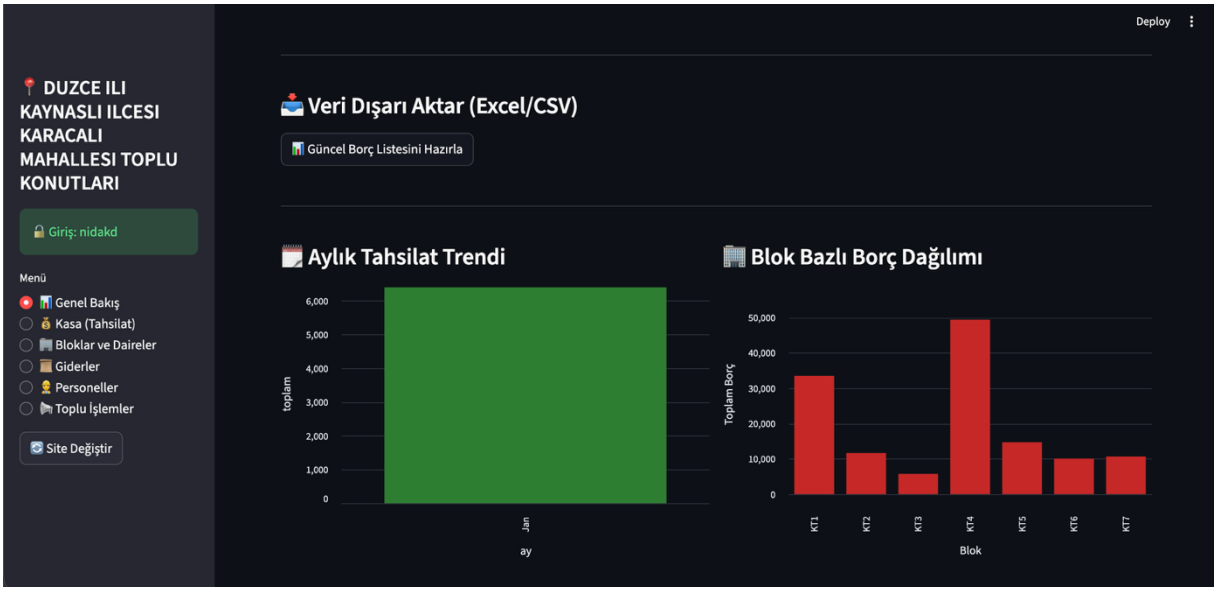


Figure 3 & 4: Management Dashboard and Financial Metrics

5.3. Block-Based Financial Monitoring and Dynamic Listing

This module, located on the application's main dashboard, enables the manager to visualize the complex organizational structure of the site systematically. When a specific block is selected, the system triggers background **SQL JOIN** queries to dynamically list all units within that block, along with resident information and real-time balance statuses.

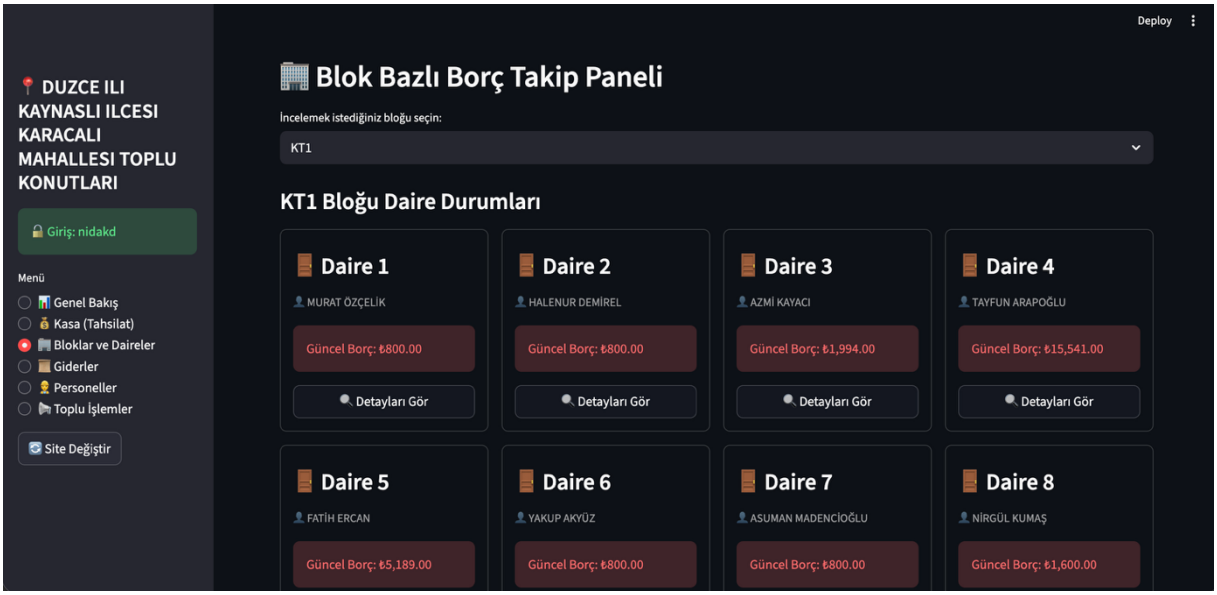
5.3.1. Block-Specific Bulk Debt Analysis

Unlike standard listing interfaces, this section provides an instantaneous calculation of the **total financial burden**associated with the selected block. When a manager selects a block, the system aggregates the outstanding debts of all units in that building and displays the "Total Block Debt" prominently at the top of the screen. This feature facilitates block-based budget management and allows for the immediate identification of buildings with lower payment performance.

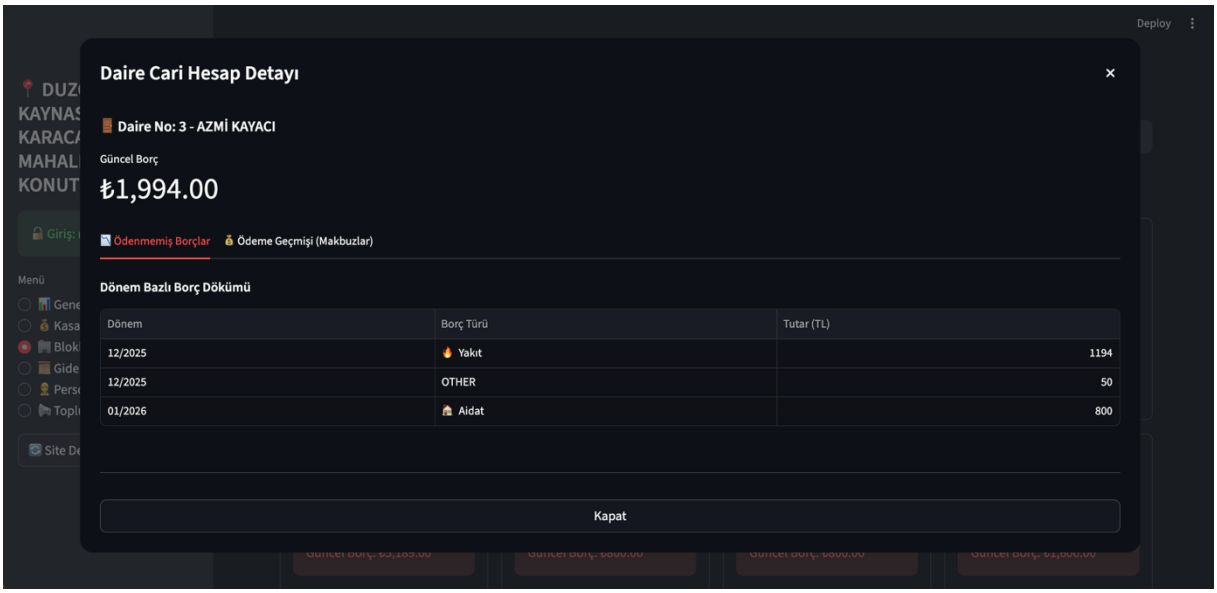
5.3.2. Detailed Account Details and Data Transparency

The "**Detail**" button on each unit's row opens a comprehensive modal window that reveals the entire financial history of the specific unit. This window is designed with two main functional axes to ensure data accuracy:

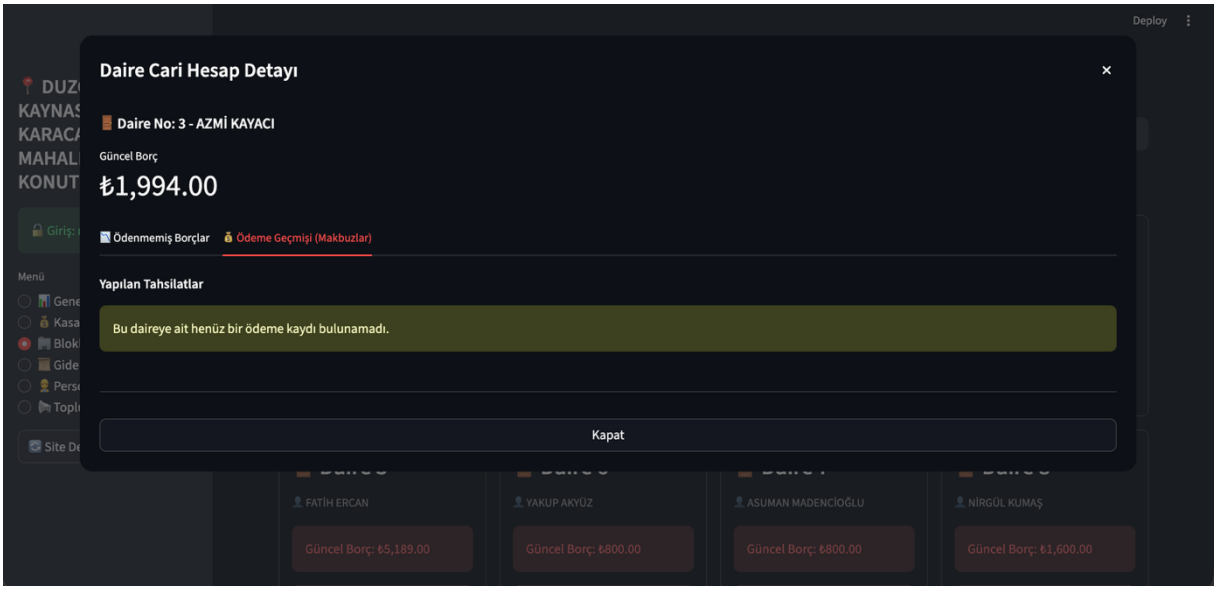
- **Unpaid Debts:** Lists every individual dues (aidat), fuel, and additional payment item since the unit's record creation. For each item, the due date, debt category, and precise amount are retrieved in real-time from the database.
- **Payment History (Receipt Records):** All collections and payments made by the resident are listed in chronological order. This section serves as **digital evidence** for the manager, providing a transparent audit trail in case of disputes or objections.



5.3.1



5.3.2



5.3.2

Figure 5, 6, and 7: Block-Based Bulk Debt Display and Unit Account Statement

5.4. Unit Details and Collection Procedures (Dialog)

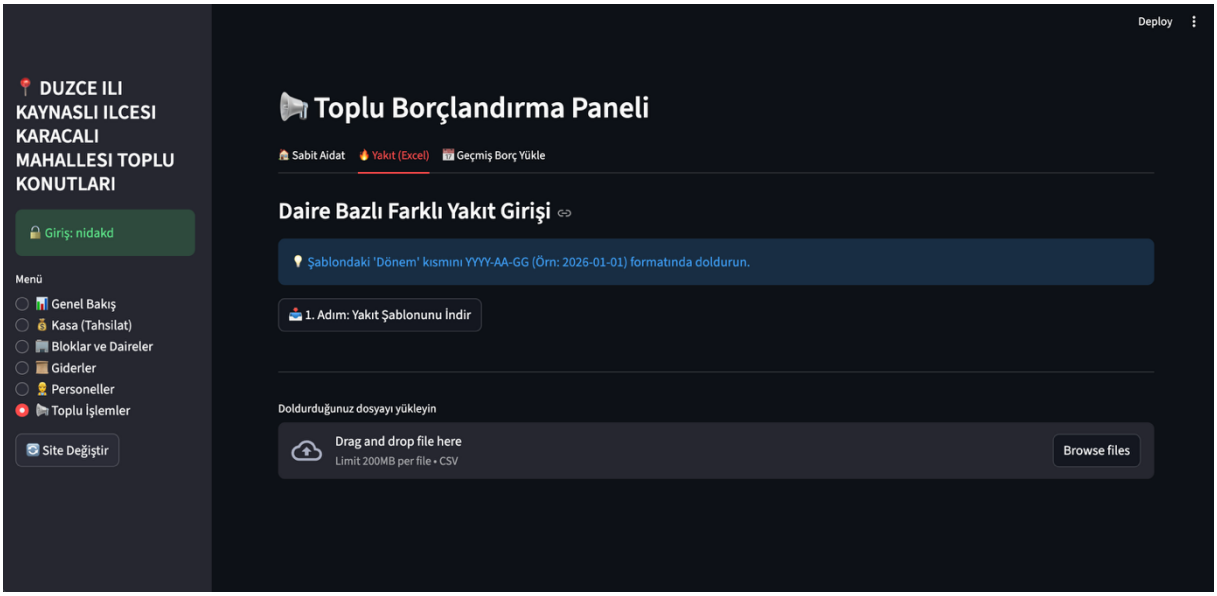
The `@st.dialog` module is triggered when the "**Detail**" button for any unit is selected. This interface is organized into two functional tabs to streamline the user experience:

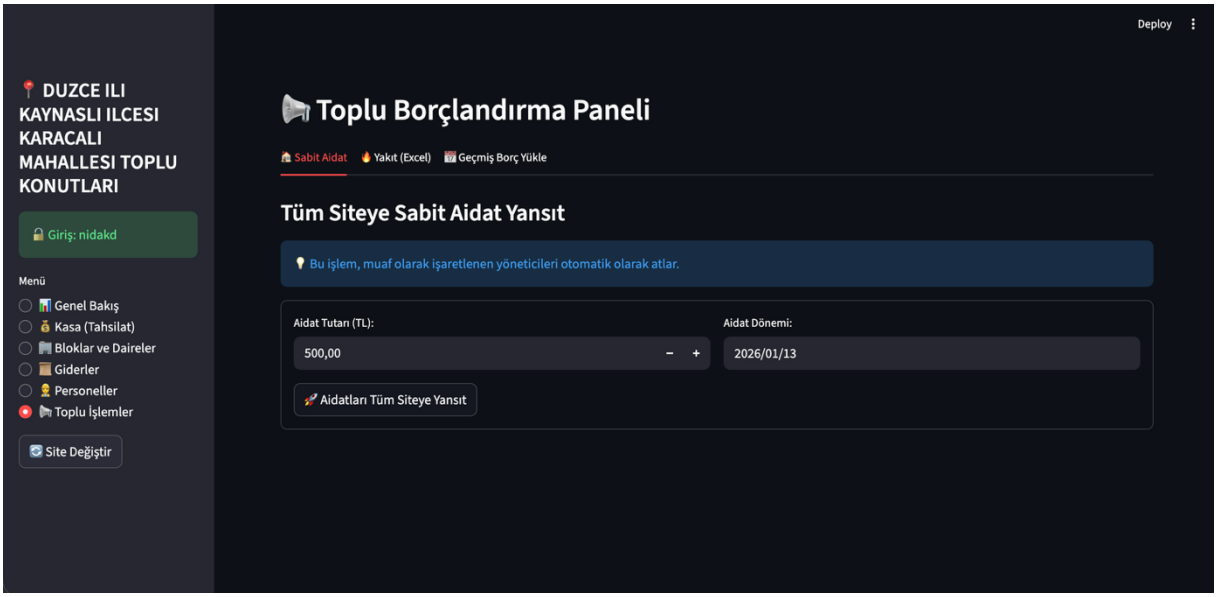
1. **Statement:** This tab provides a comprehensive list of all paid and unpaid debt items associated with the unit, offering a clear financial snapshot.
2. **Make Payment:** This is the primary interface for recording collections. When a manager enters a payment amount, the system's **FIFO (First-In, First-Out) Algorithm** automatically allocates the funds to the oldest outstanding debts first. This ensures that arrears are settled in chronological order without requiring manual selection from the manager.

Figure 8: Unit Details and Smart Collection Module

5.5. Bulk Debt Loading and CSV Integration

This module allows managers to migrate data typically maintained in Excel spreadsheets directly into the digital system.
The `toplu_yakit_yukle` function analyzes the uploaded CSV file and utilizes the **Pandas** library to parse the data. It automatically synchronizes the unit numbers with their corresponding unique IDs in the database, enabling the creation of hundreds of debt records simultaneously. This automation eliminates the need for manual data entry and ensures high data accuracy across the site.





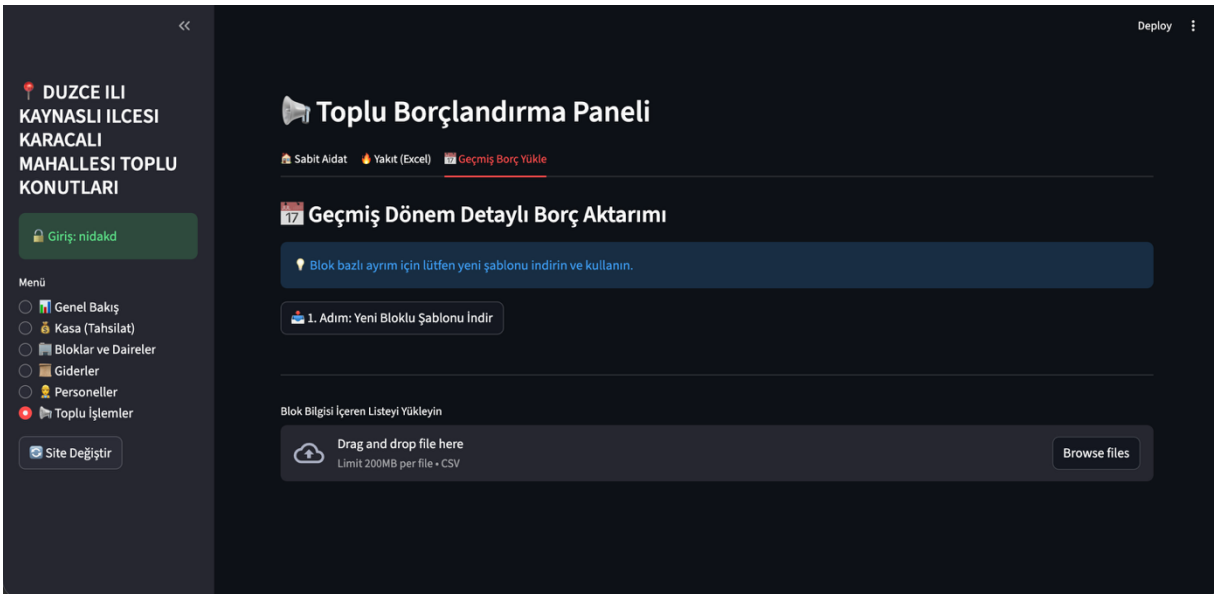


Figure 9, 10, and 11: CSV Data Migration and Bulk Debt Management Panel

5.6. Personnel Management and Salary Payment Module

This module is designed to maintain comprehensive records of employees who manage the operational tasks of the site and to execute financial obligations toward personnel (salary payments) within a digital environment. Far beyond being a simple registration list, it serves as an active **payroll management panel**.

- **Personnel Information Repository:** Personal and professional data, including names, job descriptions, contact details, and salary amounts, are managed through this interface. This module, fully integrated with the `staff` table in the database, ensures that employee records are stored systematically.
- **Active Salary Payment Function:** Utilizing the dedicated payment interface, the manager can select an employee and execute a "**Salary Payment**" transaction. Upon completion, the system automatically performs the following actions:
    1. Generates a personnel-specific payment record.
    2. Automatically logs the transaction into the site's general "**Expenses**" ledger.
    3. Instantly deducts the paid amount from the "**Cash Balance**" metric on the main dashboard to maintain real-time financial accuracy.
- **Financial Analysis and Filtering:** The manager can track payment histories for each employee individually. With role-based filtering, the monthly costs for security, cleaning, or technical staff can be analyzed on a single screen, facilitating better budgetary oversight.

Figure 12 & 13: Personnel Management and Payroll Tracking Interface

5.7. Expense Management and Cash Flow Analysis

This module manages the financial outflow of the application, centralizing salary payments and all external expenditures (invoices, maintenance, repairs, etc.) into a unified tracking system.

- **Integrated Expense Recording:** Salary payments processed through the Personnel Module are automatically categorized under "Personnel Expenses" within this list. This synchronization ensures that data is entered from a single source and propagates throughout the entire system, maintaining strict **Data Integrity**.
- **Categorical Tracking:** General expenditures such as electricity, water, and elevator maintenance are reported in separate categories alongside personnel costs. This provides a transparent documentation of "where and how much" the site management is spending, allowing for better fiscal oversight.

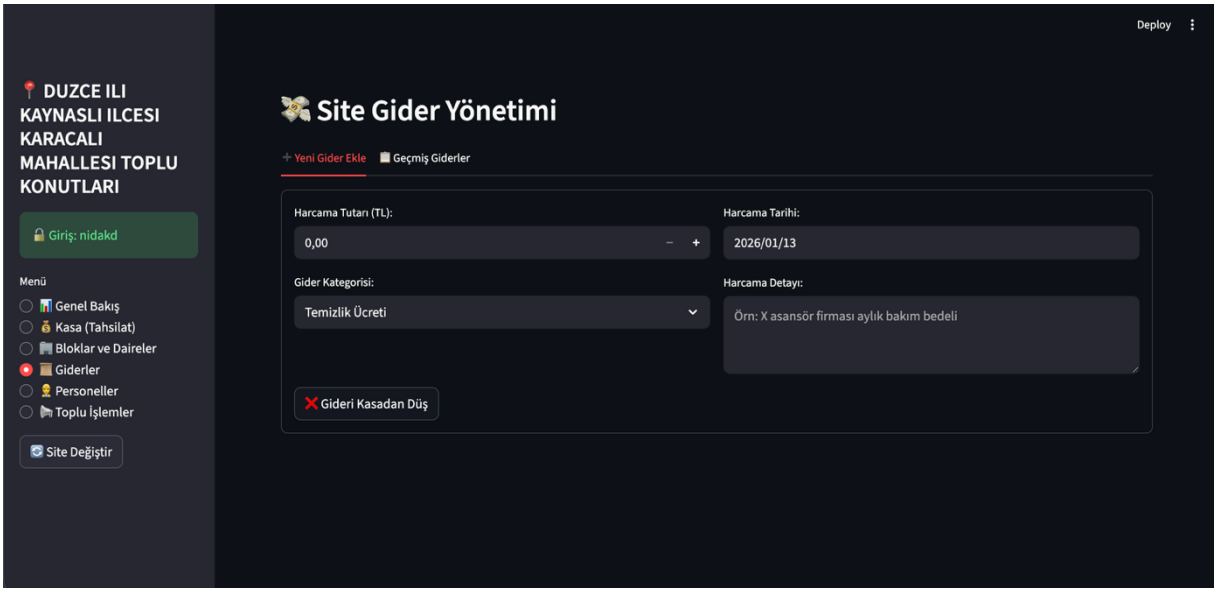


Figure 14: Expense Management and Cash Flow Analysis Interface

5.8. Reporting and Data Exporting

After completing all financial operations—including debts, collections, personnel payroll, and general expenditures—this module enables the manager to generate comprehensive reports to be presented to the **audit committee** or site residents.

- **CSV and Data Export Capability:** Integrated download buttons within the application convert real-time database tables into **UTF-8 encoded CSV files**. This ensures that data remains portable and accessible across different platforms.
- **Archiving and Compliance:** These digital records can be opened with spreadsheet tools like Microsoft Excel for further analysis or converted into physical printouts. This functionality facilitates the matching of digital transactions with **legal ledger records**, ensuring the management remains compliant with official auditing standards.

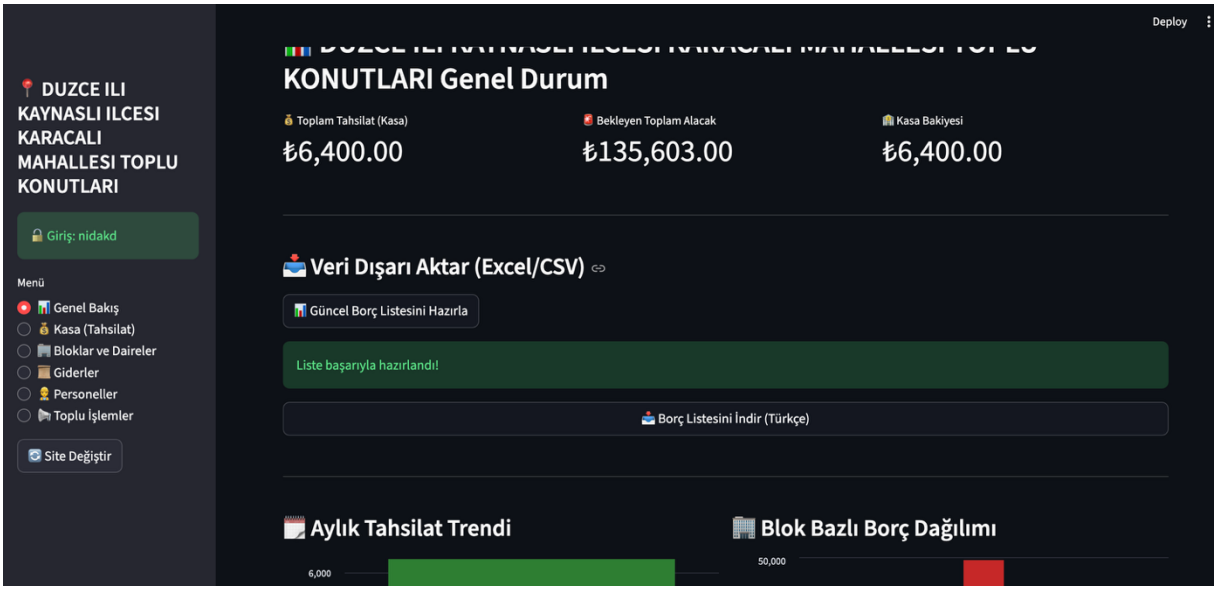


Figure 15: Reporting and Data Exporting Interface

CHAPTER 6: INSTALLATION AND RUNNING GUIDE

This section contains the technical steps required to run the application from scratch on a local server or developer computer.

6.1. System Requirements

The following environment must be ready for the application to work stably:

- **Python 3.10 or higher:** For application logic and libraries.
- **PostgreSQL 14+:** For database management and SQL functions.
- **Modern Web Browser:** Google Chrome, Firefox, or Edge (For interface viewing).

6.2. Step-by-Step Installation

1. **Installing Dependencies:** All necessary libraries must be installed via the terminal with the command:

   ```
   pip install streamlit psycopg2-binary pandas python-dotenv
   ```

2. **Database Configuration:** A new database for the project must be created on PostgreSQL, and the shared SQL scripts (Tables, Triggers) must be executed in order via the Query Tool.
3. **Environment Variables:** An `.env` file must be created, and database username/password information must be entered into this file.
4. **Starting the Application:** The following command must be entered in the terminal in the project directory:

   ```
   streamlit run main.py
   ```

CHAPTER 7: CONCLUSION AND EVALUATION

Within the scope of this graduation project, a comprehensive ERP (Enterprise Resource Planning) solution has been developed to digitize site management processes carried out by traditional and manual methods. The work done has succeeded in gathering all financial and operational processes needed by an apartment/site management onto a single platform.

7.1. Achievements and Technical Outputs

Concrete results obtained throughout the project process and added values brought to the system are presented below:

- **Digital Transformation and Data Security:** Risks of loss, destruction, or incorrect entry that may occur in physical ledger records have been eliminated with a PostgreSQL-based relational database architecture. Managing sensitive data with a `.env` file ensured the system's compliance with modern security standards.
- **Financial Accuracy (FIFO Algorithm):** Thanks to the FIFO (First-In-First-Out) based automatic collection mechanism, one of the most original outputs of the project, balance confusion caused by human error has been prevented. Automatically deducting payments from the oldest overdue debt has maximized financial transparency.
- **Operational Efficiency:** Thanks to the developed smart CSV data transfer module, manual bulk debt entry processes that could take hours have been reduced to seconds. This allowed the manager to focus on strategic decisions by reducing the operational burden.
- **Integrated Financial Management:** The fact that personnel salary payments, general expenses, and cash balance work in a connected (integrated) manner proved the technical consistency of the system as accounting software.

7.2. Future Work

This developed prototype has a sustainable infrastructure and is suitable for expansion with the following features in later stages:

- **Mobile Application Integration:** Adding a mobile interface where unit owners can see their own debts and make payments via credit card.
- **Automation:** Automatically creating e-invoices or digital receipts after collection and transmitting them to residents via e-mail/SMS.
- **AI-Supported Analysis:** Forecasting future expenses in light of past data and optimizing the budget planning module.

CHAPTER 8: BIBLIOGRAPHY AND REFERENCES

Academic publications, technical documentation, and software libraries listed below were utilized during the research, design, and development phases of this project.

8.1. Academic Sources and Books

1. **Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019).** *Database System Concepts (7th Edition).* McGraw-Hill Education. (Used as a primary reference in relational database design, normalization, and SQL query optimization processes).
2. **Lutz, M. (2013).** *Learning Python (5th Edition).* O'Reilly Media. (Consulted for object-oriented programming and Python data structures architecture).
3. **Sommerville, I. (2015).** *Software Engineering (10th Edition).* Pearson. (Provided methodological guidance in software life cycle and modular system design phases).

8.2. Technical Documentation and Web Resources

4. **PostgreSQL Global Development Group (2024).** *PostgreSQL 16.0 Documentation.* Access: https://www.postgresql.org/docs/16/index.html (For Trigger functions and Transaction management).
5. **Streamlit Inc. (2024).** *Streamlit API Reference Guide.* Access: https://docs.streamlit.io (For interface components, `@st.dialog`, and `st.session_state` management).
6. **Python Software Foundation (2024).** *Python Language Reference, v3.11.* Access: https://docs.python.org/3/.

8.3. Software Libraries Used

7. **Pandas (v2.x):** Used in the phase of data analysis and processing CSV files by converting them into DataFrame structures. https://pandas.pydata.org/
8. **Psycopg2 (v2.9):** Used to provide a secure and high-performance connection (driver) between the Python programming language and the PostgreSQL database. https://www.psycopg.org/
9. **Python-dotenv (v1.0):** Used to securely transfer sensitive data within the `.env` file (`DB_PASS`, `DB_USER`, etc.) to the application environment.
10. **OS Module:** The Python standard library was utilized for system-level directory management and access to environment variables.

8.4. Methodological References

11. **FIFO (First-In, First-Out) Accounting Principle:** The "First In, First Out" principle used in accounting standards was taken as a basis in the mathematical modeling of the collection algorithm.
12. **ER (Entity-Relationship) Modeling:** ER modeling techniques were applied in schematizing the one-to-many relationships between database tables.